
AWS Jam 学习与复盘手册

AWS Jam 八题整合索引

使用目标

1. 看到报错先判断它属于路径、权限、网络、版本还是资源策略问题
2. 进入控制台后迅速定位到真正要改的那一项，而不是在界面里盲找
3. 赛后把一次次操作回收到服务机制，形成可迁移的做题能力

覆盖范围	8 份 AWS Jam 文档，按主题合并重排
阅读方式	初学建立地图，复盘回收原理，考前速刷查漏
重点结构	模块扉页、原题侧栏、复习卡片、附录索引
编排日期	2026 年 3 月 9 日

阅读顺序

1. 先看模块扉页和题目地图
2. 再看每题的原题侧栏与最小修复动作
3. 最后用速查表和附录做考前回看

先分辨问题域，再进入控制台。

阅读地图

1	如何使用这本手册	3
1.1	先按这四种方式来读	3
1.2	页面结构怎么读	3
1.3	三轮复习法	4
1.4	AWS Jam 排障心法	4
1.5	七类高频问题域	4
1.6	题目地图	5
1.7	八份源题如何映射到本手册	6
2	模块一：数据库与数据治理	9
2.1	RDS 备份与恢复	9
2.2	RDS Graviton 迁移	11
2.3	Aurora 集群与 Failover	12
2.4	RDS 审计日志导出到 CloudWatch	13
2.5	CloudWatch Logs 数据保护与 PII 遮蔽	14
3	模块二：Serverless、网络与共享代码	18
3.1	VPC Lambda 为什么在公有子网也不能上网	18
3.2	Lambda 无法读写 S3 对象	19
3.3	Lambda Layers 与共享依赖	21
4	模块三：DevOps 交付与流水线	24
4.1	CodePipeline 找不到 CloudFormation 模板	24
4.2	CodePipeline 中并行加入 Lint	26
4.3	ECR 仓库策略挡住了 CodeBuild	27
5	模块四：安全策略与跨账号控制	30
5.1	KMS 管理员被锁出	30
5.2	S3 跨账号访问为什么会放大到整个对方账号	31
6	考前复习区	34

6.1	考前 10 分钟怎么刷	34
6.2	高频误区清单	34
6.3	症状-原因-修复速查表	34
6.4	赛后复盘模板	35
6.5	最后的学习建议	36
A	源题附录索引	37
A.1	按源文件回看	37
A.2	按题型回看	37
B	原题 PDF 附录	38
B.1	ARM64 Your DataBase.	39
B.2	Who messed up the data in my DB Cluster	46
B.3	Aws jam cloudwatch pii.	52
B.4	Look before you leap on the Cloud.	65
B.5	Sharing is caring - reusable code across Lambdas	80
B.6	CodePipeline Not Working.	87
B.7	NEED FOR SPEED!!!	98
B.8	Protect Your ECR Image.	105

如何使用这本手册

1.1 先按这四种方式来读

阅读提示	
第一次学	先看模块导读和题目地图，建立服务之间的连接，不急背着每个按钮。
第二次练	跟着最小修复动作复现，重点记控制台入口和哪一项配置最常出错。
赛后复盘	把每道题压缩成“症状 → 根因 → 修复 → 原理”。
考前速刷	直接看复习卡片、误区清单和症状-原因-修复速查表。

1.2 页面结构怎么读

版块	你要关注什么	适合什么时候看
模块扉页	这一组题的判断边界、覆盖源题和先看重点	每进入一个新模块时先看
原题侧栏	原题资源名、验收信号、控制台入口和源文件位置	刷题、回放原题、二次演练
最小修复动作	现场该怎么改，改之前先看哪类信号	刷题、排障、二次演练
底层原理	为什么这题要这样改，它真正考的服务机制是什么	赛后复盘、迁移到新题
高频误区	最容易把题做偏、做重、做慢的判断错误	考前纠偏、查漏补缺
复习卡片	一句带走的记忆锚点，适合快扫	临考前、零散时间回看

1.3 三轮复习法

轮次	关注点	你要做什么	产出
第一轮	服务关系与题型分类	识别题目属于数据库、Serverless、DevOps 还是安全治理	一张题目地图
第二轮	控制台入口与最小修复动作	只记“在哪改”和“改哪一项”，先别背长段概念	一份操作清单
第三轮	现象与原理绑定	把每个题目压缩成“症状 → 根因 → 修复 → 原理”	一套复习卡片

1.4 AWS Jam 排障心法

底层原理

1. 先看原始报错，不要先猜，更不要先大改。
2. 先判问题域：**路径、权限、网络、版本/兼容性、资源策略、Endpoint、生命周期**。
3. 只改最小必要项，避免把题目从“修配置”做成“重建环境”。
4. 每改完一次都回到验证信号，例如 Pipeline 状态、日志是否有新数据、角色是否恢复访问。
5. 赛后必须把动作上升为原理，否则下次你只会记住 UI，不会记住机制。

1.5 七类高频问题域

问题域	第一眼信号	先查什么	典型题型
路径	“file not found” “template not exist”	Artifact 名称、相对路径、工作目录	CodePipeline 模板找不到
权限	403、AccessDenied、被锁出	身份策略、资源策略、Trust Policy	S3 访问、KMS 管理、ECR 推送

问题域	第一眼信号	先查什么	典型题型
网络	超时、连接不上公网或私网目标	子网类型、路由表、NAT/IGW、Security Group	Lambda 出网、VPC 内部访问
版本/兼容性	控制台里没有可选项、运行时报兼容错误	引擎版本、架构支持矩阵、运行时	Graviton、Layer 兼容性
资源策略	明明有 Allow 仍被拒绝	显式 Deny、Principal、Condition	ECR、S3 跨账号
Endpoint	服务看起来切换了，但应用没断或还在走旧入口	Cluster Endpoint、Reader Endpoint、DNS 抽象层	Aurora Failover
生命周期	恢复、保留、遮蔽是否追溯历史	自动清理、版本不可变、新旧数据边界	RDS 备份、Layer、Logs masking

1.6 题目地图

模块	主题	核心服务	复习提示
数据库与数据治理	RDS 备份、Graviton、Aurora Failover	RDS, Aurora	先分清“恢复”和“切换”
数据库与数据治理	RDS 审计日志上云	RDS, CloudWatch Logs	记住 Option Group + 审计插件 + Log exports
数据库与数据治理	CloudWatch PII 遮蔽	CloudWatch Logs	记住 managed data identifiers 与 logs:Unmask
Serverless 与网络	VPC Lambda 出网	Lambda, VPC, NAT Gateway	记住 Lambda ENI 没有公网 IP
Serverless 与网络	Lambda 访问 S3	Lambda, IAM, S3	桶 ARN 和对象 ARN 分开记
Serverless 与网络	Lambda Layers	Lambda, S3	记住版本不可变、同函数不能挂同一 Layer 两个版本

模块	主题	核心服务	复习提示
DevOps 与交付	CodePipeline 模板路径故障	CodePipeline, CodeCommit, CloudFormation	先看 Artifact 里的相对路径
DevOps 与交付	CodePipeline 并行 Lint	CodePipeline, CodeBuild	同 Stage 同 runOrder 并行
DevOps 与交付	ECR 推送被拒	ECR, IAM, CodeBuild	看 Deny 是否把服务角色也拦掉了
安全治理	KMS 管理员锁出	KMS, IAM	Key Policy 先于 IAM 生效
安全治理	S3 跨账号过宽	S3, IAM	:root 是把权限委派给整个账号

1.7 八份源题如何映射到本手册

源文件	对应主题	补进来的关键细节	复习时怎么用
ARM64 Your DataBase.tex	RDS / Aurora	手动快照可跨账号共享、保留期为 0 关闭自动备份、实例类映射规律、Failover 约 30 秒	看“恢复 / 切主 / 改规格”边界
Who messed up the data in my DB Cluster.tex	RDS 审计	Option Group 与主版本绑定、QUERY 事件、日志组命名路径	记审计的完整配置链路
Aws jam cloudwatch pii.tex	CloudWatch 遮蔽	Task 1 修策略、Task 2 先识别字段、区域后缀 identifier、logs:Unmask	记“先识别再遮蔽”
Look before you leap on the Cloud.tex	网络 / 权限 / 安全	NAT 路由、S3 ARN 粒度、KMS 锁出、:root 语义	复习高频误判
Sharing is caring - reusable code across Lambdas.tex	Lambda Layers	/opt 路径、版本不可变、灰度升级、目录结构错误	记 Layer 生命周期
CodePipeline Not Working.tex	模板路径故障	GenerateChangeSet 修改点、SourceApp:: 语法、Jam 保存选项	记“先看工件再看模板”

源文件	对应主题	补进来的关键细节	复习时怎么用
NEED FOR SPEED!!!.tex	并行 Lint	Analysis 阶段、 runOrder=1、 SourceArtifact、 LintProject	记并行语义
Protect Your ECR Image.tex	ECR 策略	ArnNotEquals + aws:PrincipalArn 逻辑、上传镜像的附加权限	记“保留 Deny，排除合法角色”

01

模块一：数据库与数据治理

恢复、切主、审计、遮蔽

模块扉页

覆盖源题

ARM64、DB Cluster 审计、CloudWatch PII

先看边界

恢复 vs 切主；审计记录什么 vs 日志展示什么；历史日志 vs 新日志

本模块高频入口

RDS Databases / Snapshots / Option groups；CloudWatch Log groups

考前先背

PITR 属于自动备份；Failover 不改连接串；masking 只影响新日志

模块一：数据库与数据治理

模块导读	
整合来源	ARM64、DB Cluster 审计、CloudWatch PII
你会遇到	恢复、切主、迁移、审计和日志遮蔽，这一组题最怕把“动作入口”和“机制边界”混掉。
先分辨	恢复是不是基于备份重建；切主是不是在现有集群内部换 Writer；审计和遮蔽是不是分别作用在“记录什么”和“展示什么”。
高频入口	RDS → Databases / Snapshots / Option groups；CloudWatch → Log groups。
考前只背这三件事	PITR 属于自动备份；Aurora 切主不等于恢复；Logs masking 只影响新写入日志。

2.1 RDS 备份与恢复

原题侧栏	
原题资源	RDS 实例、自动备份窗口、DB snapshots
验收信号	能区分 PITR 和 snapshot restore，并知道恢复会新建实例
原题入口	RDS → Databases → Modify / Actions
源文件	ARM64 Your DataBase.tex

最小修复动作	
场景	题目要求你恢复数据库、比较备份机制，或判断能否做时间点恢复。

生命周期 / 备份策略

先判问题域

RDS → Databases / Snapshots / Restore to point in time

控制台入口

- 自动备份支持 **PITR（时间点恢复）**，保留期为 1-35 天
- **Backup retention period** 设为 0 代表关闭自动备份，生产环境通常不建议
- **Backup window** 最好放在业务低峰时段
- 手动快照需手动创建，也需手动删除，适合长期保留
- 无论是快照恢复还是时间点恢复，**都会创建新实例**，不会覆盖原实例

底层原理

不是按钮顺序，而是自动备份和手动快照在生命周期上的差异。

真正要记住的

维度	自动备份	手动快照
时间点恢复	支持	不支持
保留策略	到期自动清理	永久保留直到手动删除
跨账号共享	一般不作为共享载体	可共享，适合交接或长期留档
删除实例后	通常随实例生命周期结束	可继续保留
使用场景	运维恢复、误操作回滚	发布前留档、长期归档

高频误区

- 把“恢复”理解成覆盖原实例，结果在题里一直找“回滚按钮”
- 看到“长期留存”仍然去选自动备份，而不是手动快照

复习卡片

一句带走

看到“恢复到某个时刻”先想到自动备份和 PITR；看到“长期保留”先想到手动快照。

2.2 RDS Graviton 迁移

原题侧栏

原题资源

当前 x86 RDS 实例、目标 Graviton 实例类，例如 `db.r5.large` → `db.r6g.large`

验收信号

控制台能选到兼容的 g 实例类，修改后实例恢复 **Available**

原题入口

RDS → Databases → Modify → DB instance class

源文件

ARM64 Your DataBase.tex

最小修复动作

场景

题目要求把 x86 的 RDS 实例迁到 ARM 架构，以提升性价比。

先判问题域

版本 / 架构兼容性

控制台入口

RDS → Databases → Modify → DB instance class

- 在 **Modify** 页面查看当前实例规格和同档位可选家族
- 选择同级别、同内存档位的 Graviton 家族规格
- Jam 场景通常选 **Apply immediately**，便于立刻验证

底层原理

核心判断

Graviton 能否选择，不是死记某一张版本表，而是看 **实例家族**、**引擎主版本** 和 **官方支持矩阵**。

- 常见家族包括 r6g/m6g、r7g/m7g、r8g/m8g
- 迁移时通常保持 vCPU 和内存档位不变，例如 `r5.large` → `r6g.large`
- 控制台能否出现该实例类，往往比你记忆中的旧表更可靠
- 这类变更通常伴随重启；若是 Multi-AZ，停机影响通常会比单可用区更小

高频误区

- 把“架构代际”和“引擎最低版本”当成永远不变的固定答案
- 只看 CPU 架构，不看实例类是否在当前版本下真的可选

复习卡片

一句带走

Graviton 题先看控制台可选实例类，再回到官方支持矩阵确认，不要背过时表。

2.3 Aurora 集群与 Failover

原题侧栏

原题资源

Aurora Cluster、Writer / Reader 实例、Cluster Endpoint

验收信号

目标 Reader 升为 Writer，Cluster Endpoint 仍可用，应用连接串无需修改

原题入口

RDS → Databases → Aurora Cluster → Actions → Failover

源文件

ARM64 Your DataBase.tex

最小修复动作

场景

题目要你把某个 Reader 提升为 Writer，或者解释为什么切主后应用不用改连接串。

先判问题域

Endpoint / 集群抽象

控制台入口

RDS → Databases → 进入 Aurora Cluster → Actions → Failover

- 先进入 **Aurora Cluster**，不要只盯某个实例页面
- 通过 **Failover** 提升目标 Reader
- 验证 Writer 身份是否切换，以及 Cluster Endpoint 是否仍可用

底层原理

- **Cluster Endpoint** 指向当前 Writer
- **Reader Endpoint** 负载均衡到只读实例
- 手动或自动 Failover 通常在 **30 秒左右** 完成，原 Writer 会自动降级为 Reader
- Failover 的价值在于：**切主不改应用连接地址**

高频误区

- 把 Failover 和 Restore 混为一谈，一个是集群内换主，一个是基于备份重建
- 只验证实例角色变化，不验证应用实际连接的 Endpoint

复习卡片

一句带走

“恢复”是重建；“Failover”是重选 Writer。应用不改连接串的关键在于 Cluster Endpoint 把切换细节藏掉了。

2.4 RDS 审计日志导出到 CloudWatch

原题侧栏

原题资源

RDS 实例 challenge-node-777、自定义 Option Group、audit 日志组

验收信号

CloudWatch 中出现 audit 日志组并持续写入，必要的查询事件可见

原题入口

RDS → Option groups; RDS → Databases → Modify → Log exports

源文件

Who messed up the data in my DB Cluster.tex

最小修复动作

场景

题目要求开启 MySQL 审计并把日志送到 CloudWatch，用来追踪谁改了数据库。

先判问题域

功能入口 / 日志导出

控制台入口

RDS → Option groups; RDS → Databases → Modify → Log exports

1. 创建自定义 Option Group，数据库引擎和 Major version 要与实例一致
2. 添加 MARIADB_AUDIT_PLUGIN
3. 让 SERVER_AUDIT_EVENTS 包含查询事件，例如 QUERY 或 QUERY_DML
4. 记住一个高频参数：SERVER_AUDIT_QUERY_LOG_LIMIT=1024
5. 在实例 **Modify** 页面关联 Option Group，并勾选 **Audit log** 导出

底层原理

- Option Group 用来给 RDS 引擎挂额外能力，default 组不能直接修改
- 只配 CONNECT 只能看到登录登出，看不到 SQL 文本
- 配置生效后通常会开始记录，但官方仍提示该变更**可能造成短暂中断**
- CloudWatch 中常见日志组路径是 /aws/rds/instance/< 实例名 >/audit

高频误区

- 只勾了 Log exports，却没装审计插件
- 把 Parameter Group 和 Option Group 的职责混掉，结果一直找不到正确入口

复习卡片**一句带走**

这题的固定组合是 Option Group + 审计插件 + Log exports + CloudWatch 日志组。

2.5 CloudWatch Logs 数据保护与 PII 遮蔽

原题侧栏**原题资源**

sensitive-data-01-lambda / sensitive-data-02-lambda 日志组

验收信号

新写入日志中的 PII 被遮蔽；若具备权限，可用 logs:Unmask 做核验

原题入口

CloudWatch → Logs → Log groups → Data protection

源文件

Aws jam cloudwatch pii.tex

最小修复动作

场景

Lambda 日志打印了邮箱、手机号、SSN、密钥等敏感字段，需要在 CloudWatch Logs 里做 masking。

日志展示边界 / 数据保护

先判问题域

CloudWatch → Logs → Log groups → Data protection

控制台入口

1. 若题目像原始 Task 2 一样未知字段类型，先打开最新 Log stream 识别具体 PII
2. 打开目标 Log Group 的 **Data protection**
3. 选择需要的 managed data identifiers
4. 如需审计，再配置 S3、CloudWatch Logs 或 Firehose 作为 audit destination
5. 保存后用**新日志**验证遮蔽是否生效

底层原理

- 遮蔽只对**新写入日志**生效，不会追溯历史日志
- 如果手写 JSON，identifier 名称必须与官方 managed data identifier ID 完全一致
- 一些 identifier 带区域后缀，例如 PhoneNumber-US、Ssn-US、DriversLicense-US
- 默认看到的是遮蔽后的内容；有 logs:Unmask 的主体才能查看未遮蔽数据
- 仅 **Standard log class** 支持 masking
- 若配置审计目标，还需要额外的 logs:CreateLogDelivery 或目标服务权限

高频误区

- 反复看旧日志，以为配置没生效
- Jam 里只要求“让遮蔽生效”，却把时间花在手写复杂 JSON 上

复习卡片**一句带走**

Masking 考的是“新旧日志边界”与“谁能看未遮蔽内容”，不是纯勾选题。

02

模块二：Serverless、网络与共享代码

网络、权限、打包、版本

模块扉页

覆盖源题

Look before you leap、Lambda Layers

先看边界

超时多半是网络；403 多半是权限；`ImportModuleError` 多半是 Layer 或打包结构

本模块高频入口

Lambda Configuration / Layers；VPC Route tables；IAM Roles；S3

考前先背

VPC Lambda 出网靠私有子网 + NAT；对象操作要写 `/*`；Layer 版本不可变

3

模块二：Serverless、网络与共享代码

模块导读

整合来源	Look before you leap、Lambda Layers
你会遇到	Lambda 这组题最容易把网络、权限和打包问题混成一团。
先分辨	超时多半是网络；403 多半是权限； <code>ImportModuleError</code> 多半是打包或 Layer 结构。
高频入口	Lambda → Configuration / Layers；VPC → Subnets / Route tables；IAM → Roles；S3。
考前只背这三件事	VPC Lambda 出网靠私有子网 + NAT；S3 桶 ARN 和对象 ARN 分两层；Layer 版本不可变。

3.1 VPC Lambda 为什么在公有子网也不能上网

原题侧栏

原题资源	目标 Lambda、私有子网、NAT Gateway、CloudWatch Logs、VPC 内 EC2 Web 服务
验收信号	重试后超时消失，函数既能访问私网目标，也能访问公网资源
原题入口	Lambda → Configuration → VPC；VPC → Route tables / NAT Gateways
源文件	Look before you leap on the Cloud.tex

最小修复动作

场景	Lambda 被放进 VPC 后访问互联网超时，但访问 VPC 内私有资源正常。
----	--

网络 先判问题域	
控制台入口	Lambda → Configuration → VPC; VPC → Route tables / NAT Gateways
	• 把 Lambda 放入私有子网 • 确保该子网默认路由到 NAT Gateway • 不要把“子网被标成 Public”误当成 Lambda 可以直接上公网 • 同时确认安全组放行对外 80/443，以及访问私有 EC2 所需端口
底层原理	
	• Lambda ENI 不会拿到公网 IP • Internet Gateway 需要公网地址映射才能完成对外访问 • 所以“公有子网 + IGW”对 EC2 可行，对 Lambda 不成立
高频误区	
	• 看到 Public Subnet 就下意识觉得一定能上网 • 只看 Security Group，不看路由表里到底是 0.0.0.0/0 → nat-* 还是 igw-*
复习卡片	
一句带走	VPC Lambda 想上互联网，关键词不是 Public Subnet，而是 Private Subnet + NAT Gateway。

3.2 Lambda 无法读写 S3 对象

原题侧栏	
原题资源	目标 Lambda 执行角色、Bucket Policy / IAM Policy、对象 ARN
验收信号	GetObject / PutObject 不再 403，Lambda 可正常读写对象
原题入口	IAM → Roles → Permissions / Trust relationships; S3 → Bucket policy

源文件

Look before you leap on the Cloud.tex

最小修复动作

场景

Lambda 角色已经有 S3 权限，但 `GetObject`、`PutObject` 仍然 403。

先判问题域

权限 / ARN 粒度

控制台入口

IAM → Roles → 目标执行角色；S3 → Bucket policy；Lambda → Execution role

- 桶级权限用 `arn:aws:s3:::bucket-name`
- 对象级权限必须用 `arn:aws:s3:::bucket-name/*`
- 若题目还包含删除动作，别漏掉 `s3:DeleteObject`
- Trust Policy 的服务主体需要是 `lambda.amazonaws.com`

底层原理

S3 ARN 有两层语义

桶本身和桶内对象不是同一个资源面。

ARN

对应权限

`arn:aws:s3:::my-bucket`

桶本身，例如 `s3:ListBucket`

`arn:aws:s3:::my-bucket/*`

桶里的对象，例如
`s3:GetObject`、`s3:PutObject`

高频误区

- 给了桶 ARN，却忘了对象 ARN
- 一直改权限动作列表，却发现 Trust Policy 主体写错
- 在 CloudFormation 里直接复用桶 ARN，却没写成 `!Sub "$MyBucket.Arn/*"`

复习卡片

一句带走

S3 403 先问自己两件事：对象 ARN 有没有 /*，角色能不能被 Lambda 正常扮演。

3.3 Lambda Layers 与共享依赖

原题侧栏

原题资源

tattooine-common layer、ChallengeFunctionOne / Two、task-one / task-two layer zip

验收信号

Runtime.ImportModuleError 消失；FunctionOne 可切到 v2，FunctionTwo 保持 v1 做灰度

原题入口

Lambda → Layers；Lambda → Functions → Add a layer

源文件

Sharing is caring - reusable code across Lambdas.tex

最小修复动作

场景

两个 Lambda 缺共享依赖，报 Runtime.ImportModuleError。

先判问题域

打包结构 / 运行时兼容性

控制台入口

Lambda → Layers；Lambda → Functions → Add a layer

1. 从 S3 创建自定义 Layer
2. 按原题约束检查兼容架构与运行时，例如 x86_64、Node.js 22.x
3. 把 Layer 挂到目标函数
4. 发布新版本时，先升级一个函数做灰度验证

底层原理

- Layer 只适用于 **.zip 部署** 函数，不适用于容器镜像函数
- 版本不可变，更新必须发新版本
- 一个函数最多关联 5 个 Layer

- 同一个函数不能同时引用同一 Layer 的两个版本
- 关联后内容会被解压到 `/opt/`
- Node.js 最通用的目录是 `/opt/nodejs/node_modules`

高频误区

- 目录结构打错，尤其是 Node.js 依赖没有放在 `nodejs/node_modules` 这类可识别路径
- 想“直接覆盖”旧 Layer 版本，而不是发新版本再替换引用

复习卡片

一句带走

Node.js Layer 最稳的目录是 `nodejs/node_modules`；Layer 的本质是“只读版本化共享依赖”。

03

模块三：DevOps 交付与流水线

路径、并行语义、资源策略

模块扉页

覆盖源题

Pipeline 路径、并行 Lint、ECR 策略

先看边界

找不到模板先查路径；时间变长先查 Stage 与 runOrder；推镜像被拒先查显式 Deny

本模块高频入口

CodePipeline Edit pipeline；CodeBuild Service role；ECR Repository policy

考前先背

先看 Artifact；同 Stage 同 runOrder 才并行；显式 Deny 最高优先

模块三：DevOps 交付与流水线

模块导读	
整合来源	Pipeline 路径、并行 Lint、ECR 策略
你会遇到	这一组题看起来像“流水线坏了”，但真正原因通常分别是路径、并行语义和资源策略。
先分辨	找不到模板先查路径；耗时变长先查 Stage 和 runOrder；推镜像被拒先查显式 Deny。
高频入口	CodePipeline → Edit pipeline；CodeBuild → Service role；ECR → Repository policy。
考前只背这三件事	Artifact 路径写相对路径；同 Stage 同 runOrder 并行；显式 Deny 最高优先。

4.1 CodePipeline 找不到 CloudFormation 模板

原题侧栏	
原题资源	SourceApp 工件、DeployTestStack、ChangeSet Action、目标模板路径
验收信号	Pipeline 成功执行；新的 CloudFormation 堆栈创建成功；堆栈名出现在主堆栈 Outputs
原题入口	CodePipeline → Edit pipeline → Deploy action → Template path
源文件	CodePipeline Not Working.tex

最小修复动作	
场景	Deploy 阶段报错：

```
File [template.yaml] does not exist in artifact [SourceApp]
```

路径

先判问题域

CodePipeline → Edit pipeline → Deploy action → Template path

控制台入口

- 原题里真正修改的是 DeployTestStack 阶段中的 GenerateChangeSet Action
- 先确认 Source Artifact 中真实存在的相对路径
- 再修改 CloudFormation Action 使用的模板路径
- 在本 Jam 环境下，保存时勾选 “No resource updates needed for this source action change”，出现短暂 ValidationError 也不必先慌

底层原理

不是 CloudFormation 语法，而是 “工件名 + 相对路径” 这对组合。

这题本质

某些界面会把它显示为类似：

```
SourceApp::infrastructure-as-code/working.template.yaml
```

在 Jam 环境里也可以回到 Artifact 桶，下载 zip 后看内部真实路径。

如果仓库权限受限

高频误区

- 看到 Deploy 报错就先怀疑模板内容，忽略了路径
- 把 Jam 环境里的保存选项，当成所有生产环境都必须照搬的规则

复习卡片

Pipeline 找不到模板，先定位 Artifact，再定位相对路径。

一句带走

4.2 CodePipeline 中并行加入 Lint

原题侧栏

原题资源

Analysis Stage、SourceArtifact、LintProject-xxx、CodePipeline 预建 IAM 角色

验收信号

Lint 与现有分析动作并行执行，流水线总时长不按串行相加

原题入口

CodePipeline → Edit stage → Analysis Stage → Action runOrder

源文件

NEED FOR SPEED!!!.tex

最小修复动作

场景

现有 Pipeline 想加 Lint，但又不想串行拉长总时长。

先判问题域

并行语义

控制台入口

CodePipeline → Edit stage → Analysis Stage → Action runOrder

- 把 Lint Action 放进现有 Analysis Stage
- 让它和已有分析 Action 使用相同的 runOrder，原题里都是 1
- 常见配置组合是输入构件 SourceArtifact、项目 LintProject-xxx、Build type 为单次构建
- 保存后观察 Stage 总耗时是否由最慢 Action 决定

底层原理

- 同一 Stage、相同 runOrder 的 Action 会并行执行
- 这样能避免把多个分析步骤按串行相加
- 但 Stage 结束时间仍然取决于**最慢的那个 Action**

高频误区

- 新建一个独立 Stage，结果无意中把流程变串行
- 误以为“并行”就等于对总时长没有任何影响

- 忘了保存时勾选 Jam 环境里的“无需资源更新”选项，导致额外权限报错

复习卡片

一句带走 这题不是“怎么写 Lint”，而是“怎么利用 Stage 并行语义”。

4.3 ECR 仓库策略挡住了 CodeBuild

原题侧栏

原题资源	pyei-challenge-app、CodeBuild Service Role ARN、DenyPutImage 语句
验收信号	仓库策略中的 Deny 仍保留，但 CodeBuild 重新可以推镜像
原题入口	CodeBuild → Service role; ECR → Repositories → Permissions → Repository policy
源文件	Protect Your ECR Image.tex

最小修复动作

场景	ECR 仓库里有一条 Deny，导致 CodeBuild 也无法推镜像。
先判问题域	资源策略
控制台入口	CodeBuild → Service role; ECR → Permissions → Repository policy
	• 先找到 CodeBuild Service Role ARN • 在 ECR 仓库策略里保留 Deny • 通过 Condition + ArnNotEquals + aws:PrincipalArn 排除该服务角色

底层原理

- 显式 Deny 优先级最高
- 但可以通过条件让某些请求不满足这条 Deny

- `ecr:PutImage` 管的是镜像 manifest 和 tag，镜像层上传还依赖其他上传类动作

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "ecr:PutImage",
  "Condition": {
    "ArnNotEquals": {
      "aws:PrincipalArn": "arn:aws:iam::<account-id>:role/codebuild-xxx-
        -service-role"
    }
  }
}
```

高频误区

- 一股脑把 Deny 删掉，虽然过题，但失去原题想保留的防护意图
- 用成 `ArnEquals`，结果反而只把 CodeBuild 自己拒掉
- 只放行 `ecr:PutImage`，却忘了镜像推送还需要上传层相关动作

复习卡片

一句带走

Deny 不是一定要删，而是要让合法的服务角色从条件里被排除出去。

04

模块四：安全策略与跨账号控制

权限入口、Principal 粒度、最小授权

模块扉页	
覆盖源题	Look before you leap
先看边界	KMS 先看 Key Policy 入口；S3 跨账号先看 Principal 是否写得过宽
本模块高频入口	KMS Key policy；S3 Bucket policy；IAM Roles
考前先背	Key Policy 先于 IAM；:root 代表整个账号；跨账号要同时看资源策略与身份策略

5

模块四：安全策略与跨账号控制

模块导读

整合来源	Look before you leap
你会遇到	这组题最考验对资源策略边界的理解，不是会不会点按钮，而是有没有看清“谁先生效、把门开给谁”。
先分辨	KMS 看 Key Policy 入口；S3 跨账号看 Bucket Policy 的 Principal 是否写得过宽。
高频入口	KMS → Key policy；S3 → Bucket policy；IAM → Roles / Policies。
考前只背这三件事	KMS 先看 Key Policy；:root 代表整个账号；跨账号访问要同时看资源策略和对方身份策略。

5.1 KMS 管理员被锁出

原题侧栏

原题资源	jam-data-encryption-key、tmp admin role、JamTask3SecurityAdminRoleArn
验收信号	管理员角色重新回到 KeyAdministrators 中，后续可以继续管理该 Key
原题入口	KMS → Customer managed keys → Key policy
源文件	Look before you leap on the Cloud.tex

最小修复动作

场景	你是 IAM 管理员，但突然无法继续管理某个 KMS Key。
----	---------------------------------

权限入口 / Key Policy

先判问题域

KMS → Customer managed keys → Key policy

控制台入口

- 先切换到仍有权限的临时管理员角色，例如原题里的 tmp admin role
- 编辑 Key Policy，把可靠的管理员主体重新加回去
- 至少保留两个稳定管理员，避免再次单点锁出

底层原理

- KMS 的权限入口是 **Key Policy**
- IAM Policy 不是天然生效，而是要先被 Key Policy 放行
- 如果既没有账户级入口，也没有其他管理员主体，Key 可能变成 **unmanageable**
- 严重情况下可能需要更高权限管理员甚至 AWS Support 介入

高频误区

- 把 KMS 当成普通 IAM 服务，以为有管理员权限就能接管一切
- 修复时只加回一个人，留下新的单点风险

复习卡片

一句带走

KMS 题先问“我是不是在 Key Policy 入口里”，而不是先问“IAM Policy 写没写”。

5.2 S3 跨账号访问为什么会放大到整个对方账号

原题侧栏

原题资源

目标 Bucket、EXTERNAL_ACCOUNT_ID、role/S3CrossAccountAccess

验收信号

只有 S3CrossAccountAccess 角色可访问，其他主体会被拒绝

原题入口

S3 → Bucket policy

Look before you leap on the Cloud.tex

源文件

最小修复动作

场景

你本来只想允许外部账号里的某个角色访问桶，但实际上整个外部账号的人都能用。

先判问题域

资源策略 / Principal 粒度

S3 → Bucket policy

控制台入口

把 Bucket Policy 的 Principal 从：

```
arn:aws:iam::EXTERNAL_ACCOUNT_ID:root
```

改成精确的角色 ARN，例如：

```
arn:aws:iam::EXTERNAL_ACCOUNT_ID:role/S3CrossAccountAccess
```

底层原理

:root 在跨账号资源策略里并不表示“只允许对方 Root 用户”，而是把权限委派给整个外部账号。之后对方账号管理员还能再通过 IAM 把权限授给更多主体。

高频误区

- 看到 :root 就直觉理解成“只放给对方根用户”
- 只看自己这一侧的 Bucket Policy，不看对方账号还能如何二次分发权限
- 忘记对敏感数据再加 MFA、IP 或 VPC Endpoint 等条件限制

复习卡片

一句带走

跨账号访问要记两层：资源策略决定把门开给谁，对方身份策略决定谁最终能穿过这扇门。

05

考前复习区

速刷、速查、复盘模板

模块扉页

适合什么时候看

考前 10 分钟、做题间隙、赛后统一回收

先看顺序

先扫问题域，再刷误区清单，最后看速查表和复盘模板

本区用途

把前面的题目压缩成能快速回忆的最小信息单元

6

考前复习区

6.1 考前 10 分钟怎么刷

阅读提示

1. 先看“七类高频问题域”，把题目归到路径、权限、网络、版本、资源策略、Endpoint 或生命周期。
2. 再看各模块的“模块导读”，回想每组题最容易混掉的判断边界。
3. 最后只刷复习卡片和下面的速查表，不再重新读完整解释。

6.2 高频误区清单

高频误区

- 把 Public Subnet 当成 Lambda 出网的充分条件
- 把 S3 桶 ARN 和对象 ARN 混为一谈
- 把 KMS 的权限模型当成普通 IAM 服务理解
- 把 CodePipeline 的模板路径问题误判成模板语法问题
- 把并行执行误解成“永远零耗时增加”
- 把 CloudWatch PII identifier 名称手写错
- 把 :root 误读成“只允许对方 Root 用户”
- 把 Failover 和 Restore 混成同一类动作

6.3 症状-原因-修复速查表

症状	根因	修复动作	底层原理
Lambda 访问互联网超时	放在公有子网，只有 IGW 路由	改到私有子网并走 NAT Gateway	Lambda ENI 没公网 IP

症状	根因	修复动作	底层原理
S3 对象操作 403	对象 ARN 少了 /* 或 Trust Policy 不对	补对象级资源 ARN，并确认执行角色可被 Lambda 扮演	桶级和对象级 ARN 分离，角色信任关系先成立
KMS 无法管理 Key	当前主体不在 Key Policy 入口内	用仍有权限的角色修 Key Policy	KMS 先看 Key Policy
Pipeline 找不到模板	Artifact 中相对路径写错	改模板路径字段	先定位工件，再定位路径
加 Lint 后时间明显变长	串行新增 Stage 或最慢 Action 变慢	同 Stage 同 runOrder 并行	并行不等于无限加速
ECR 推镜像被拒	仓库策略 Deny 没排除服务角色	用 Principal ARN 条件排除	显式 Deny 优先
CloudWatch 看不到遮蔽	配错 identifiers 或看的是历史日志	选对 identifiers 并写入新日志验证	masking 只影响新日志
Aurora 切主后应用不改串	应用连的是 Cluster Endpoint	验证 Writer 身份而非改连接地址	Endpoint 层屏蔽主从变化
恢复题要求某时刻回滚	误把快照当成 PITR 手段	走自动备份时间点恢复	PITR 属于自动备份能力

6.4 赛后复盘模板

阅读提示

每做完一道 AWS Jam 题，都建议只用下面 5 行做复盘：

1. **症状：**我最先看到的报错或异常现象是什么
2. **对象：**真正出问题的是哪一个资源、哪一条策略、哪一条路径
3. **修复：**我最终只改了哪一项
4. **原理：**这背后对应的 AWS 机制是什么
5. **防复发：**如果在生产环境，应该加什么保护措施

6.5 最后的学习建议

底层原理

不是重复看操作截图，而是重复做这三件事。

真正有效的复习

- 把每道题压缩成一句“症状-原因-修复”
- 把每种修复动作映射回 AWS 控制台入口
- 把每个入口再映射回权限、网络、存储或生命周期原理

当你能在脑中先分辨问题域，再进入控制台找入口时，AWS Jam 的速度和稳定性都会明显上来。

A

源题附录索引

A.1 按源文件回看

源文件	本手册对应页	回看主题	高频资源
ARM64 Your DataBase.tex	9 / 11 / 12	RDS 备份、Graviton、Aurora Failover	RDS 实例、Snapshots、Aurora Cluster
Who messed up the data in my DB Cluster.tex	13	RDS 审计日志上云	challenge-node-777、audit group
Aws jam cloudwatch pii.tex	14	CloudWatch Logs PII 遮蔽	sensitive-data-01 / 02
Look before you leap on the Cloud.tex	18 / 19 / 30 / 31	VPC Lambda、S3、KMS、跨账号 S3	子网路由、执行角色、KMS Key、Bucket Policy
Sharing is caring - reusable code across Lambdas.tex	21	Lambda Layers 共享依赖	tattooine-common、Challenge-Functions
CodePipeline Not Working.tex	24	CloudFormation 模板路径故障	SourceApp、ChangeSet Action
NEED FOR SPEED!!!.tex	26	CodePipeline 并行 Lint	Analysis Stage、SourceArtifact、LintProject
Protect Your ECR Image.tex	27	ECR 仓库策略修复	pyei-challenge-app、CodeBuild role

A.2 按题型回看

题型	先回看哪一节	再回哪份源题
恢复 / 备份	第 9 页附近的 RDS 备份与恢复	ARM64 Your DataBase.tex
架构迁移 / 切主	第 11、12 页附近	ARM64 Your DataBase.tex
日志审计 / 遮蔽	第 13、14 页附近	Who messed up the data in my DB Cluster.tex、 Aws jam cloudwatch pii.tex
Lambda 网络 / 权限	第 18、19 页附近	Look before you leap on the Cloud.tex
共享依赖 / Layer 升级	第 21 页附近	Sharing is caring - reusable code across Lambdas.tex
Pipeline 路径 / 并行	第 24、26 页附近	CodePipeline Not Working.tex、NEED FOR SPEED!!!!.tex
资源策略 / 显式 Deny	第 27、31 页附近	Protect Your ECR Image.tex、Look before you leap on the Cloud.tex
KMS 管理入口	第 30 页附近	Look before you leap on the Cloud.tex

B

原题 PDF 附录

B.1 ARM64 Your DataBase

对应手册

RDS 备份、Graviton 迁移、Aurora Failover

适合回看

需要回看备份差异、实例类映射、切主流程时

PDF 文件

ARM64 Your DataBase.pdf

AWS Jam 知识点整理

Amazon Aurora RDS & Graviton 实践

2026 年 3 月 9 日

目录

Amazon RDS 备份机制

两种备份类型

RDS 提供两种互补的备份机制：

特性	自动备份	手动快照
触发方式	自动（每日）	手动触发
保留期限	1-35 天后自动删除	永久保留，需手动删除
时间点恢复	✓支持	× 不支持
跨账号共享	×	✓支持
删除实例后	自动删除	继续保留

自动备份配置

- 路径：RDS → Databases → 实例 → **Modify** → Backup
- **Backup retention period**：设置保留天数（建议生产环境 ≥ 7 天）
- **Backup window**：建议选择业务低峰时段
- 设为 0 则关闭自动备份（不推荐生产环境使用）

手动快照操作

控制台路径：RDS → Databases → Actions → **Take snapshot**

CLI 创建快照

```
aws rds create-db-snapshot \  
  --db-instance-identifier my-db \  
  --db-snapshot-identifier my-snapshot-20260309
```

恢复备份

- 恢复操作会**创建新实例**，而非覆盖原实例
- 快照恢复：Snapshots → Actions → **Restore snapshot**
- 时间点恢复：实例详情 → Actions → **Restore to point in time**

Graviton CPU 实例

什么是 Graviton

AWS Graviton 是亚马逊自研的 **ARM 架构**处理器，RDS 支持以下几代：

- **Graviton2**: r6g、m6g、c6g 系列
- **Graviton3**: r7g、m7g 系列
- **Graviton4**: r8g、m8g 系列

相较 x86 实例，性价比提升约 **20–35%**。

引擎版本要求

Graviton 可用性取决于具体实例家族、数据库引擎、主版本与次版本，不适合写成一张固定的总表。

- 不同代际的最低版本不同：例如较新的 m8g/r8g 通常要求比 m7g/r7g 更高的引擎次版本
- 同一引擎也会分家族：是否能选某个 Graviton 型号，应以 RDS 控制台 **Modify** 页面可选项或官方 **DB instance class support matrix** 为准
- 迁移前先核对矩阵：不要假设 “MySQL 8.0+ / PostgreSQL 12+ / MariaDB 10.4+” 对所有 Graviton 家族都成立

实例类型对应关系

更换 Graviton 时，需保持 **vCPU 和内存不变**，仅更换 CPU 架构：

x86 实例	Graviton 对应实例	规格
db.r5.large	db.r6g.large	2 vCPU / 16 GB
db.r5.xlarge	db.r6g.xlarge	4 vCPU / 32 GB
db.m5.large	db.m6g.large	2 vCPU / 8 GB
db.m5.xlarge	db.m6g.xlarge	4 vCPU / 16 GB

规律：系列字母不变 ($r \rightarrow r$, $m \rightarrow m$)，数字后加 g，大小规格不变。

修改实例操作

1. RDS → Databases → 选中实例 → **Modify**
2. Instance configuration → DB instance class → 选择 Graviton 型号
3. 生效时间选择 **Apply immediately** (Jam 实验需立即验证)
4. 确认修改

CLI 修改实例类型

```
aws rds modify-db-instance \  
  --db-instance-identifier my-db \  
  --db-instance-class db.r6g.large \  
  --apply-immediately
```

注意：更换实例类型需要重启；Multi-AZ 部署可将停机时间压缩至秒级。

Amazon Aurora 集群架构与故障转移

Aurora 集群基本结构

- 一个集群包含一个 **Writer 实例**（主节点）和若干 **Reader 实例**（从节点）
- **Cluster Endpoint**: 始终指向当前 Writer，应用无需感知主从变化
- **Reader Endpoint**: 负载均衡所有 Reader，用于只读查询

Failover（故障转移）

Failover 是将指定 Reader 提升为 Writer 的标准方式，特点如下：

- 切换时间约 **30 秒内**完成
- 原 Writer 自动降级为 Reader，**无需手动干预**
- 应用连接串不需要修改（Cluster Endpoint 自动更新）
- 可随时再次 Failover 回滚，适合灰度测试场景

Failover 操作步骤

1. RDS → Databases → 点击进入 **Aurora 集群**
2. 右上角 **Actions** → **Failover**
3. 在弹出框中选择目标实例（要提升为 Writer 的实例）
4. 确认执行
5. 等待状态恢复为 **Available** 后验证结果

Failover 与主从手动切换的区别

方式	停机时间	适用场景
Failover（手动触发）	~30 秒	计划内主从切换、灰度测试
自动 Failover（故障时）	~30 秒	主节点宕机时自动触发
Modify 实例类型	重启时间	变更实例规格

本次 Jam 操作流程总结

Task 1. RDS 备份：了解自动备份与手动快照的差异，掌握配置和恢复操作。

Task 2. 更换 Graviton 实例：

- 查看当前实例类型（如 `db.r5.large`）
- 找到规格相同的 Graviton 对应型号（如 `db.r6g.large`）
- 通过 Modify 更改 DB instance class

Task 3. Aurora 主从切换：

- 通过 Failover 将 Graviton 实例提升为 Writer
- 验证写操作是否路由到 Graviton 实例
- 如需回滚，再次执行 Failover 恢复 x86 为 Writer

整理于 2026 年 3 月 9 日

B.2 Who messed up the data in my DB Cluster

对应手册

RDS 审计日志上云

适合回看

需要核对 Option Group、审计参数或日志组路径时

PDF 文件

Who messed up the data in my DB Cluster.pdf

AWS Jam Challenge

Task 1：启用 RDS MySQL 审计日志并导出至 CloudWatch

知识点总结 | 操作步骤 | 核心概念

2026 年 3 月 9 日

任务背景

任务描述

Amazon RDS MySQL 实例 `challenge-node-777` 未启用审计日志（Audit Logging / Option Group），导致无法对数据库操作进行追踪与取证。

目标：在 RDS MySQL 上启用审计日志（通过 Option Group 配置 MARIADB_AUDIT_PLUGIN），并将审计日志推送至 Amazon CloudWatch。

涉及服务

服务名称	作用	备注
Amazon RDS MySQL	数据库实例	<code>challenge-node-777</code>
Option Group	插件配置容器	承载 MARIADB_AUDIT_PLUGIN
MARIADB_AUDIT_PLUGIN	审计插件	记录连接与查询事件
Amazon CloudWatch	日志存储与查询平台	接收审计日志流

核心知识点

什么是 Option Group？

Option Group 是 Amazon RDS 的一种配置容器，用于为数据库引擎启用额外功能插件。每个 RDS 实例可关联一个 Option Group，通过在其中添加 Option 来

扩展数据库能力。

Option Group 关键概念

- 每个 Option Group 绑定特定的数据库引擎和版本（如 MySQL 8.0）
- 一个实例同时只能关联一个 Option Group
- 修改 Option Group 后，部分选项需要重启实例才能生效
- AWS 提供的 default Option Group 不可修改，需创建自定义 Option Group

什么是 MARIADB_AUDIT_PLUGIN?

MARIADB_AUDIT_PLUGIN 是 RDS MySQL 支持的审计插件，用于记录数据库的连接事件与查询事件，并将日志写入文件或 CloudWatch。

参数名	说明	推荐值
SERVER_AUDIT_LOGGING	是否开启审计	ON
SERVER_AUDIT_EVENTS	记录的事件类型	CONNECT, QUERY
SERVER_AUDIT_QUERY_LOG_LIMIT	单条查询日志长度限制（字节）	1024
SERVER_AUDIT_INCL_USERS	仅审计指定用户（空 = 全部）	按需设置
SERVER_AUDIT_EXCL_USERS	排除指定用户	可排除 rdsadmin

注意

SERVER_AUDIT_EVENTS 需要包含能记录 SQL 文本的事件类型，例如 QUERY 或 QUERY_DML，才能捕获 DML 操作。

若仅设置 CONNECT，则只记录登录/登出事件，无法记录 SQL 语句。

CloudWatch 日志导出

RDS 支持将以下类型的日志直接推送至 CloudWatch Logs：

- **Audit log**（审计日志）← 本任务目标
- Error log（错误日志）
- General log（通用日志）
- Slow query log（慢查询日志）

日志组命名规则：

```
1 /aws/rds/instance/<实例名>/audit
2 # 本任务对应：
3 /aws/rds/instance/challenge-node-777/audit
```

操作步骤

Step 1: 创建自定义 Option Group

1. 登录 AWS 管理控制台，进入 RDS 服务
2. 在左侧菜单点击 **Option groups**
3. 点击 **Create group**，填写以下信息：
 - **Name:** 自定义名称，如 my-audit-option-group
 - **Description:** 描述信息
 - **Engine:** MySQL Community
 - **Major engine version:** 与实例版本一致（如 8.0）
4. 点击 **Create** 完成创建

Step 2: 添加 MARIADB_AUDIT_PLUGIN 选项

1. 进入刚创建的 Option Group，点击 **Add option**
2. 在 **Option name** 中选择 MARIADB_AUDIT_PLUGIN
3. 配置关键参数：
 - SERVER_AUDIT_LOGGING = ON
 - SERVER_AUDIT_EVENTS = CONNECT, QUERY
 - SERVER_AUDIT_QUERY_LOG_LIMIT = 1024
4. 点击 **Add option** 保存

Step 3: 将 Option Group 关联到 RDS 实例

1. 进入 **RDS → Databases**，点击实例 challenge-node-777
2. 点击右上角 **Modify**（修改）

3. 在 **Additional configuration** 部分找到 **Option group**
4. 从下拉菜单中选择刚创建的 Option Group
5. 继续向下找到 **Log exports** 部分，勾选 **Audit log**
6. 点击 **Continue**，选择 **Apply immediately**（立即应用）
7. 点击 **Modify DB instance** 完成修改

补充说明

为 MySQL 添加 MARIADB_AUDIT_PLUGIN 后，审计通常会在 Option Group 生效后立即开始，不需要手动重启实例；但 AWS 官方仍提示该变更可能造成短暂中断，生产环境建议安排在维护窗口或低峰时段执行。

Step 4: 验证日志是否推送至 CloudWatch

1. 进入 **CloudWatch** → **Logs** → **Log groups**
2. 搜索日志组：

```
1 /aws/rds/instance/challenge-node-777/audit
```

3. 若日志组存在且有数据流入，则说明配置成功

审计日志格式说明

审计日志每行为 CSV 格式，字段含义如下：

```
1 20260309 10:37:00,ip-172-17-5-73,rdsadmin,localhost,8,493,QUERY,mysql,'SELECT 1',0,,
```

字段位置	字段名	示例值
1	时间戳	20260309 10:37:00
2	服务器主机名	ip-172-17-5-73
3	用户名 (USERNAME)	rdsadmin
4	客户端主机	localhost
5	连接 ID	8
6	查询 ID	493
7	操作类型	QUERY / CONNECT
8	数据库名	mysql
9	SQL 语句	'SELECT 1'
10	返回码	0 (成功)

常见问题排查

问题 1: CloudWatch 日志组中只有 SELECT，没有 UPDATE 记录

原因：审计日志只记录启用之后的操作，历史操作不会被追溯。

解决：等待攻击行为再次触发，或手动执行一条 UPDATE 语句进行测试。

问题 2: CloudWatch Insights 查询返回 0 条结果

可能原因：

- 时间范围选择过短（默认 1 小时），应调整为更长时间范围
- 选错了日志组，注意区分 /audit、/error、/general
- SERVER_AUDIT_EVENTS 未包含 QUERY，导致 DML 未被记录

总结

Task 1 核心要点

1. RDS MySQL 默认不开启审计，需通过自定义 **Option Group** 添加 MARIADB_AUDIT_PLUGIN 插件
2. 插件的 SERVER_AUDIT_EVENTS 至少要包含能记录 SQL 的事件类型（如 QUERY 或 QUERY_DML）；若只配 CONNECT 则看不到 SQL
3. 在 RDS 实例的 **Modify** 页面同时完成两件事：
(1) 关联自定义 Option Group (2) 勾选 Audit log 导出至 CloudWatch
4. 日志组路径固定为 /aws/rds/instance/< 实例名 >/audit
5. 审计日志为 CSV 格式，第 3 个字段为 **用户名**，可用 CloudWatch Insights 解析查询

B.3 Aws jam cloudwatch pii

对应手册

CloudWatch Logs PII 遮蔽

适合回看

需要回看 Task 1 / Task 2 差别、identifier 或审计权限时

PDF 文件

Aws jam cloudwatch pii.pdf

AWS Jam 实验总结

CloudWatch 日志数据保护策略

本文档涵盖 AWS CloudWatch 日志组
PII（个人可识别信息）数据保护的
题目背景、操作步骤与核心知识点

服务：Amazon CloudWatch | 难度：入门

2026 年 3 月 9 日

目录

1 背景与概述

1.1 实验背景

在企业 AWS 环境中，Lambda 函数常将运行日志发送至 Amazon CloudWatch Logs。若开发人员在代码中不慎将用户隐私字段（如姓名、邮箱、身份证号、信用卡号等）直接打印到日志，将导致 **PII**（**Personally Identifiable Information**，个人可识别信息）泄露，违反 GDPR、CCPA 等数据合规要求。

 说明

PII 是指任何能够单独或结合其他信息识别特定个人的数据，包括姓名、电子邮件地址、社会安全号、信用卡号、IP 地址等。

1.2 实验目标

本次 AWS Jam 包含两个关联任务，核心目标一致：

- 识别 CloudWatch 日志组中暴露的 PII 类型
- 为对应日志组配置正确的数据保护策略（Data Protection Policy）
- 使策略生效，对敏感字段进行自动遮蔽（Masking）

1.3 涉及 AWS 资源

资源类型	资源名称	说明
CloudWatch Log Group	/aws/lambda/sensitive-data-01-lambda	任务一：已有错误保护策略
CloudWatch Log Group	/aws/lambda/sensitive-data-02-lambda	任务二：需先识别 PII 类型

2 任务一：修正错误的数据保护策略

2.1 题目描述

日志组 `sensitive-data-01-lambda` 正在记录包含 PII 信息的日志，且已配置了错误的数据保护策略，需要将其修正为正确的策略以遮蔽敏感数据。

2.2 操作步骤

2.2.1 步骤 1：导航至目标日志组

1. 登录 AWS Management Console
2. 搜索并进入 CloudWatch 服务
3. 在左侧导航栏选择 **Logs** → **Log groups**
4. 找到并点击 `/aws/lambda/sensitive-data-01-lambda`

2.2.2 步骤 2：进入数据保护配置

1. 点击日志组详情页顶部的 **Data protection** 标签
2. 若已有旧策略，选择 **Edit** 进行修改；否则点击 **Create data protection policy**

2.2.3 步骤 3：选择 PII 数据类型

在数据类型选择界面，勾选所有需要保护的 PII 类型。控制台会直接显示可选数据类型；若你手写策略 JSON，则需要使用 AWS 官方的 **managed data identifier ID**。下面列出本题最常见且名称准确的标识符（建议在 Jam 中直接控制台全选，避免漏项）：

	类别	数据类型标识符	说明
金融信息		CreditCardNumber	信用卡号
		CreditCardExpiration	信用卡有效期
		CreditCardSecurityCode	CVV / Security Code
		BankAccountNumber-US	区域相关，手写策略时需带国家代码
个人身份		Name	姓名
		EmailAddress	电子邮件地址
		Address	邮寄地址

(续表)

	类别	数据类型标识符	说明
		DateOfBirth	出生日期
		PhoneNumber-US	区域相关, 电话号需带国家代码
证件号码		Ssn-US	美国社会安全号 (SSN)
		PassportNumber-US	美国护照号
		DriversLicense-US	美国驾照号
		NationalIdentificationNumber-ES	国家身份证号示例 (需带国家代码)
网络与密钥		IpAddress	IP 地址
		AwsSecretKey	AWS Secret Access Key
		OpenSshPrivateKey	OpenSSH 私钥
		PgpPrivateKey	PGP 私钥
		PkcsPrivateKey	PKCS 私钥
		PuttyPrivateKey	PuTTY 私钥
医疗健康		HealthInsuranceCardNumber-EU	欧盟健康保险卡号
		DrugEnforcementAgencyNumber-US	DEA 药物执法编号

⚠ 注意

部分标识符是区域相关的, 必须追加 ISO 3166-1 alpha-2 代码, 例如 DriversLicense-US、Ssn-US、BankAccountNumber-US。

若直接在控制台勾选, 控制台会帮你处理这些细节; 若手写 JSON, 请以 AWS 官方 managed identifier 列表为准。

2.2.4 步骤 4: 配置审计目标 (可选)

- 可选择将审计结果发送到 S3 存储桶、另一个 CloudWatch 日志组, 或 Kinesis Data Firehose
- 如无需审计, 可跳过此步骤

2.2.5 步骤 5: 保存并验证

1. 点击 **Create / Save** 保存策略
2. 返回日志流, 查看最新日志, PII 字段应显示为遮蔽格式

Listing 1: 策略生效后的日志示例

```
# 策略生效前
email: john.doe@example.com | ssn: 123-45-6789 | ip: 192.168.1.1

# 策略生效后
email: ***** | ssn: ***** | ip: *****
```

3 任务二：识别 PII 类型并应用保护策略

3.1 题目描述

日志组 `sensitive-data-02-lambda` 暴露了多种 PII 字段，需要先主动查看日志内容，识别具体的 PII 类型，再针对性地配置数据保护策略以遮蔽全部敏感信息。

⚠ 注意

本任务与任务一的核心区别在于：需要先读取日志内容进行 PII 类型识别，再制定保护策略，而非直接套用已知配置。

3.2 操作步骤

3.2.1 步骤 1：查看日志内容，识别 PII

- 1. 进入 **CloudWatch** → **Log groups**
- 2. 点击 `/aws/lambda/sensitive-data-02-lambda`
- 3. 点击 **Log streams**，打开最新的日志流
- 4. 仔细阅读日志内容，记录所有暴露的敏感字段

3.2.2 步骤 2：PII 类型映射

根据日志中看到的内容，对应选择保护类型：

日志中出现的内容	对应 PII 类型
user@example.com 格式字符串	EmailAddress
555-123-4567 格式号码	PhoneNumber-US
123-45-6789 格式编号	Ssn-US
192.168.x.x 格式地址	IpAddress
4111-1111-1111-1111 格式号码	CreditCardNumber
人名字符串	Name
街道地址字符串	Address

3.2.3 步骤 3：应用数据保护策略

操作方式与任务一相同，参考第 2.2 节步骤 2-5，根据识别结果仅勾选对应类型，或直接全选以确保完整覆盖。

 最佳实践

在实验/竞赛场景中，若不确定日志包含哪些 PII 类型，建议**全选所有类型**，这是最保险的做法，可确保通过任务验证并满足最严格的合规要求。

在生产环境中，则应仅选择**实际业务涉及的 PII 类型**，避免过度遮蔽影响日志可用性。

4 核心知识点

4.1 Amazon CloudWatch Logs 数据保护策略

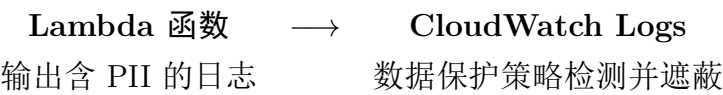
4.1.1 功能简介

CloudWatch Logs 数据保护策略（Data Protection Policy）是 AWS 提供的托管功能，可自动检测并遮蔽日志数据中的敏感信息，无需修改应用代码。

4.1.2 核心特性

- **自动检测：**基于正则模式和机器学习检测 PII
- **实时遮蔽：**新写入的日志实时进行遮蔽处理
- **不可追溯：**策略仅对新日志生效，不影响已存储的历史日志
- **审计支持：**可将检测结果输出到 S3 或其他日志组
- **权限受控：**具有特定 IAM 权限的用户可查看原始未遮蔽数据

4.1.3 工作原理



Listing 2: 遮蔽前后对比示意

```
# Lambda 实际输出
INFO: Processing user john.doe@example.com, SSN: 123-45-6789

# CloudWatch 日志中显示
INFO: Processing user *****, SSN: *****
```

4.2 所需 IAM 权限

Listing 3: 单个 Log Group 且不配置审计目标时的最小权限（JSON）

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
```

```
    "Action": [
      "logs:PutDataProtectionPolicy",
      "logs:GetDataProtectionPolicy",
      "logs>DeleteDataProtectionPolicy"
    ],
    "Resource": "arn:aws:logs:*:*:log-group:*"
  }
]
```

 说明

上述策略只覆盖单个日志组且无审计目标的最小场景。若实际操作包含以下能力，还需要额外权限：

- 配置 CloudWatch Logs 作为审计目标:还需 logs:CreateLogDelivery、logs:PutResourcePolicy、logs:DescribeResourcePolicies、logs:DescribeLogGroups
- 配置 S3 作为审计目标: 还需 logs:CreateLogDelivery、s3:GetBucketPolicy、s3:PutBucketPolicy
- 配置 Firehose 作为审计目标: 还需 logs:CreateLogDelivery，以及目标 Firehose 的相关权限
- 查看未遮蔽日志: 还需 logs:Unmask
- 创建账号级策略:还需 logs:PutAccountPolicy;删除账号级策略则需 logs>DeleteAccountPolicy

4.3 数据保护策略的局限性

局限项	说明
历史日志	策略不追溯处理已存储的历史日志
日志组类别	仅 Standard log class 支持 masking
区域限制	需在支持该功能的 AWS 区域内使用
误遮蔽风险	全选 PII 类型可能遮蔽部分非 PII 字段
查看权限	默认只能看到遮蔽后的数据；具备 logs:Unmask 的用户可查看未遮蔽内容

4.4 数据合规相关法规

- GDPR**（欧盟通用数据保护条例）：要求对个人数据进行最小化收集和保护

- **CCPA**（加州消费者隐私法）：赋予消费者对个人数据的知情权和控制权
- **HIPAA**（健康保险可携带性和责任法案）：规范医疗健康数据的处理方式
- **PCI DSS**（支付卡行业数据安全标准）：规范信用卡数据的存储与传输

5 操作速查表

✓ 最佳实践

以下为快速参考，适用于考试或竞赛场景中的快速操作。

- Step 1. 进入 CloudWatch: Console → CloudWatch → Log groups
- Step 2. 选择日志组: 点击目标 Log Group 名称
- Step 3. 查看日志 (任务二): Log streams → 最新流 → 识别 PII 字段
- Step 4. 创建策略: Actions → Create data protection policy
- Step 5. 选择类型: 全选或按需勾选 PII 数据类型
- Step 6. 保存: 点击 Create / Save
- Step 7. 验证: 返回日志流, 确认敏感字段已被星号遮蔽

本文档根据 AWS Jam 实验场景整理, 仅供学习参考。

Amazon CloudWatch 功能以 AWS 官方文档为准: docs.aws.amazon.com

B.4 Look before you leap on the Cloud

对应手册

VPC Lambda、S3 ARN、KMS、S3 跨账号

适合回看

需要统一回看网络和权限类高频误判时

PDF 文件

Look before you leap on the Cloud.pdf

AWS Jam 实战题目总结

网络、权限与安全专题

本文档汇总了四道 AWS Jam 实战题目的
问题背景、根本原因、操作步骤与知识点，
适用于 AWS 认证备考与实战参考。

涉及服务	Amazon VPC, S3, IAM, KMS, Lambda
难度	中级
类别	网络 / 权限 / 安全

2026 年 3 月 9 日

目录

1 题目一：VPC Lambda 无法访问互联网

1.1 背景

开发了一个 AWS Lambda 函数，需要同时访问：

- 互联网资源（<http://example.com>）
- VPC 内私有子网上 EC2 实例的 Web 服务

函数配置为 VPC Lambda，部署后执行成功但 CloudWatch Logs 出现网络错误。

1.2 根本原因

⚠ 根本原因

Lambda 被放置在公有子网（Public Subnet）中。

虽然公有子网的路由表指向 Internet Gateway（IGW），但 **Lambda ENI 永远不会被分配公网 IP**，IGW 找不到对应的公网 IP 映射，导致出站流量被丢弃。

1.2.1 核心原理：IGW 的工作机制

Internet Gateway 进行的是 **一对一 NAT**，要求资源拥有公网 IP：

资源类型	公有子网中能获得公网 IP	能否访问互联网
EC2 实例	✓（自动分配或 EIP）	✓
NAT Gateway	✓（必须绑定 EIP）	✓
Lambda ENI	×（AWS 不分配）	×

1.2.2 私有子网为何可行

私有子网路由表指向 NAT Gateway，NAT Gateway 自身持有 EIP，代替 Lambda 访问互联网：

```
1 # 私有子网路由表（正确）
2 Destination      Target
3 10.0.0.0/16      local
4 0.0.0.0/0        nat-xxxxxxx    <- NAT Gateway, 有公网 IP
5
6 # 公有子网路由表（对 Lambda 无效）
7 Destination      Target
8 10.0.0.0/16      local
```

```
9 0.0.0.0/0      igw-xxxxxxxx    <- IGW, Lambda ENI 无公网 IP
```

Listing 1: 私有子网路由表示例

1.3 操作步骤

✓ 解决方案

1. 打开 **CloudWatch Logs**, 确认 `/aws/lambda/MyFunction` 中存在连接超时等网络错误。
2. 打开 **Lambda Console** → 选择 `MyFunction` → **Configuration** → **VPC**。
3. 点击 **Edit**, 将子网从公有子网更换为私有子网。
 - 私有子网特征: 路由表中 `0.0.0.0/0` → `nat-*`
 - 公有子网特征: 路由表中 `0.0.0.0/0` → `igw-*`
4. 点击 **Save**, 重新执行函数并验证日志无报错。

💡 核心知识点

VPC Lambda 网络访问规则:

- Lambda 放在**私有子网** + 私有子网有 NAT Gateway 路由 → 可访问互联网 & 私有资源
- Lambda 放在**公有子网** → 只能访问私有资源, 无法访问互联网
- 安全组需放行出站 80/443 端口 (互联网) 及 EC2 对应端口

2 题目二：Lambda 无法读写 S3 对象

2.1 背景

通过 CloudFormation 创建了 S3 Bucket 和 IAM Role，用于同账号的 Lambda 函数进行 CRUD 操作。应用团队反馈 Lambda 无法对 Bucket 进行任何读写操作。

2.2 根本原因

根本原因

IAM Policy 中，对象级操作的 Resource ARN 缺少 /* 后缀。
S3 ARN 在 IAM 中具有两种语义，对象级操作必须使用 `arn:aws:s3:::bucket-name/*` 才能生效。

2.2.1 S3 ARN 两种语义对照

ARN 格式	代表含义	对应操作
<code>arn:aws:s3:::my-bucket</code>	桶本身	<code>s3:ListBucket</code> 等桶级操作
<code>arn:aws:s3:::my-bucket/*</code>	桶内所有对象	<code>s3:GetObject</code> 、 <code>s3:PutObject</code> 等

2.2.2 错误的 IAM Policy

```
1 {  
2   "Effect": "Allow",  
3   "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],  
4   "Resource": "arn:aws:s3:::my-bucket"  
5 }
```

Listing 2: 错误配置（缺少 /*）

2.2.3 正确的 IAM Policy

```
1 {  
2   "Effect": "Allow",  
3   "Action": ["s3:ListBucket"],  
4   "Resource": "arn:aws:s3:::my-bucket"  
5 },  
6 {  
7   "Effect": "Allow",
```

```
8   "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
9   "Resource": "arn:aws:s3:::my-bucket/*"
10 }
```

Listing 3: 正确配置

2.3 操作步骤

✓ 解决方案

1. 打开 **IAM Console** → **Roles** → 找到目标 Role。
2. 检查 **Trust relationships**, 确认 `lambda.amazonaws.com` 为信任主体。
3. 展开 **Permissions** 中的自定义 Policy, 找到对象级操作语句。
4. 将 **Resource** 的值从 `arn:aws:s3:::bucket-name` 修改为 `arn:aws:s3:::bucket-name/*`。
5. 保存后重新测试 Lambda 函数。

2.3.1 CloudFormation 正确写法

```
1 - Effect: Allow
2   Action:
3     - s3:ListBucket
4   Resource:
5     - !GetAtt MyBucket.Arn          # arn:aws:s3:::my-bucket
6
7 - Effect: Allow
8   Action:
9     - s3:GetObject
10    - s3:PutObject
11    - s3:DeleteObject
12   Resource:
13    - !Sub "${MyBucket.Arn}/*"      # arn:aws:s3:::my-bucket/*
```

Listing 4: CloudFormation IAM Policy 正确示例

 核心知识点

口诀：桶操作用桶 ARN，对象操作必须加 /*

IAM Policy 还需注意 Trust Policy 中服务主体正确性：

- Lambda 使用的 Role: "Service": "lambda.amazonaws.com"
- EC2 使用的 Role: "Service": "ec2.amazonaws.com"

3 题目三：KMS Key 管理员被锁出

3.1 背景

作为 AWS 安全管理员，在开发阶段创建了一个 KMS Key。开发阶段结束后，发现无法再管理该 KMS Key，且公司规定 KMS 权限仅通过 Key Policy 管理，不依赖 IAM Policy。

3.2 根本原因

⚠ 根本原因

KMS Key Policy 被修改后，当前 IAM Role 未被包含在 Key Administrators 列表中，导致失去管理权限。

KMS 的权限模型与其他 AWS 服务不同：Key Policy 是**第一道门**，IAM Policy 必须先被 Key Policy 允许才能生效。

3.2.1 KMS 权限模型对比

服务	权限控制方式	说明
S3、SQS 等	Identity-based Policy 与 Resource-based Policy 共同评估	通常取并集，显式 Deny 优先
KMS	Key Policy 是前提条件，IAM Policy 在其基础上补充	Key Policy 优先

3.2.2 被锁出的 Key Policy 示例

```
1 {
2   "Sid": "KeyAdmins",
3   "Effect": "Allow",
4   "Principal": {
5     "AWS": "arn:aws:iam::111122223333:role/dev-team-role"
6   },
7   "Action": ["kms:*"],
8   "Resource": "*"
9 }
```

Listing 5: 错误的 Key Policy（管理员角色不含当前用户）

3.3 操作步骤

✓ 解决方案

1. 在 Console 右上角切换到临时管理员角色 jam-aws-tmp-admin-role (Switch Role)。
2. 打开 **KMS Console** → 找到 jam-data-encryption-key。
3. 点击 **Key policy** 标签页 → 点击 **Edit**。
4. 在 `KeyAdministrators` 语句的 `Principal.AWS` 数组中, 添加 `JamTask3SecurityAdminRoleArn`:

```
1 {
2   "Sid": "KeyAdministrators",
3   "Effect": "Allow",
4   "Principal": {
5     "AWS": [
6       "arn:aws:iam::111122223333:role/jam-aws-tmp-admin-role",
7       "arn:aws:iam::111122223333:role/JamTask3SecurityAdminRole"
8     ]
9   },
10  "Action": [
11    "kms:Create*", "kms:Describe*", "kms:Enable*",
12    "kms:List*",   "kms:Put*",      "kms:Update*",
13    "kms:Revoke*", "kms:Disable*", "kms:Get*",
14    "kms:Delete*", "kms:ScheduleKeyDeletion", "kms:CancelKeyDeletion"
15  ],
16  "Resource": "*"
17 }
```

Listing 6: 修复后的 Key Policy

💡 核心知识点

KMS Key Policy 最佳实践:

- 永远在 Key Policy 中保留至少一个可靠的管理员角色。
- 若既未保留账户级入口, 也没有其他可管理主体, KMS Key 可能变得 **unmanageable**; 此时可能需要账户紧急管理员介入, 严重时需联系 AWS Support。

- 生产环境建议保留至少两个管理员身份，防止单点锁出。
- `kms:*` 权限不等于 IAM 的管理员权限，必须显式写入 Key Policy。

4 题目四：S3 跨账号访问控制过于宽松

4.1 背景

提供了一个 S3 Bucket 供外部 AWS 账号用户访问数据。要求仅有被分配了特定角色 S3CrossAccountAccess 的用户才能访问，但实际上该外部账号下的所有用户都能访问。

4.2 根本原因

⚠ 根本原因

Bucket Policy 中 Principal 使用了外部账号的 root，这并不表示“只允许对方 Root 用户”，而是把访问权限委派给整个外部账号。随后该账号管理员可以再通过 IAM Policy 将权限授予多个用户或角色，因此超出了“只允许特定角色访问”的要求。

4.2.1 Principal 写法的语义差异

Principal 写法	谁能访问	是否
ACCOUNT_ID:root	委派给整个外部账号，由对方账号再授予具体主体	×
ACCOUNT_ID:role/S3CrossAccountAccess	仅该特定角色	✓
ACCOUNT_ID:user/specific-user	仅该特定用户	✓

4.2.2 错误的 Bucket Policy

```

1 {
2   "Effect": "Allow",
3   "Principal": {
4     "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:root"
5   },
6   "Action": ["s3:GetObject", "s3:ListBucket"],
7   "Resource": [
8     "arn:aws:s3:::your-bucket",
9     "arn:aws:s3:::your-bucket/*"
10  ]
11 }
```

Listing 7: 错误配置（Principal 为 root）

4.2.3 修复后的 Bucket Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "AllowSpecificRoleOnly",
6       "Effect": "Allow",
7       "Principal": {
8         "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:role/
9           S3CrossAccountAccess"
10      },
11      "Action": ["s3:GetObject", "s3:ListBucket"],
12      "Resource": [
13        "arn:aws:s3:::your-bucket",
14        "arn:aws:s3:::your-bucket/*"
15      ]
16    }
17  ]
18 }
```

Listing 8: 正确配置（Principal 精确到角色）

4.3 操作步骤

✓ 解决方案

1. 打开 **S3 Console** → 找到目标 Bucket。
2. 点击 **Permissions** 标签页 → **Bucket policy** → 点击 **Edit**。
3. 将 Principal 从:
 - "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:root"修改为:
 - "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:role/S3CrossAccountAccess"
4. 点击 **Save changes**。
5. 验证：外部账号中未获得该角色权限、或未假设该角色的主体应收到 **Access Denied**；仅 **S3CrossAccountAccess** 角色可正常访问。

 核心知识点**最小权限原则 (Principle of Least Privilege):**

- 跨账号访问时, Principal 应精确到具体的 Role/User ARN。
- 避免使用 :root 进行跨账号授权; 只有在你明确要把权限委派给整个外部账号时才这样做。
- 对于敏感数据, 还应结合 Condition 加入 MFA、IP、VPC Endpoint 等限制条件。

5 综合知识点汇总

5.1 AWS 权限控制体系

服务	权限控制层	注意事项
Lambda + VPC	子网路由 (NAT/IGW)	Lambda ENI 无公网 IP
S3 + IAM	IAM Policy Resource ARN	对象操作需 /* 后缀
KMS	Key Policy 优先	须显式包含管理员 ARN
S3 跨账号	Bucket Policy Principal	避免使用 :root 委派整个账号

5.2 常见错误速查表

症状	常见原因	修复方向
Lambda 连接超时	在公有子网, 无 NAT	改为私有子网
S3 GetObject 403	IAM Resource 缺 /*	补全 ARN 后缀
KMS 无权限	Role 不在 Key Policy 中	编辑 Key Policy
跨账号访问过宽	Principal 为 root	改为精确 Role ARN

5.3 最佳实践总结

1. **VPC Lambda:** 始终将 Lambda 放置在私有子网, 通过 NAT Gateway 访问互联网。
2. **IAM Policy:** S3 对象级操作的 Resource 必须使用 `arn:aws:s3:::bucket/*` 格式。
3. **KMS Key Policy:** 始终保留至少两个管理员 ARN, 防止自我锁出。
4. **跨账号权限:** Principal 精确到角色或用户, 遵循最小权限原则。
5. **IAM 信任策略:** Lambda 角色的 Trust Policy 中服务主体必须为 `lambda.amazonaws.com`。

B.5 Sharing is caring - reusable code across Lambdas

对应手册

Lambda Layers 与共享依赖

适合回看

需要回看 Layer 版本、目录结构和灰度升级时

PDF 文件

Sharing is caring - reusable code across Lambdas.pdf

AWS Jam 实战挑战总结

Lambda Layers 共享代码库

服务: AWS Lambda | 难度: 入门 ~ 中级

挑战概览

本次 AWS Jam 挑战围绕 AWS Lambda Layers（层）展开，分为两个任务：

1. **Task 1:** 为两个 Lambda 函数添加共享代码层，解决 `Runtime.ImportModuleError` 错误。
2. **Task 2:** 发布新版本的 Layer，并将其中一个函数升级到新版本。

涉及资源：

- 共享库: `tattooine-common`（存储于 Amazon S3）
- 函数一: `LabStack-prewarm-{random}-ChallengeFunctionOne-{random}`
- 函数二: `LabStack-prewarm-{random}-ChallengeFunctionTwo-{random}`
- 兼容架构: `x86_64`
- 兼容运行时: `Node.js 22.x`（Task 2 额外支持 `Node.js 18.x`）

Task 1: 创建 Lambda Layer 并关联函数

背景与问题

两个 Lambda 函数在部署时未包含依赖库 `tattooine-common`，导致运行时抛出如下错误：

```
Runtime.ImportModuleError: Error: Cannot find module 'tattooine-common'
```

库文件已上传至 S3，路径为：

```
s3://aws-jam-challenge-resources-{REGION}/lambda-layers-shared-code/task-one/layer-task-one.zip
```

解决方案：使用 Lambda Layers

Lambda Layers 是 Lambda 提供的共享机制，用于在多个函数间复用依赖库、自定义运行时或配置文件。实际操作时，可在函数详情页的 **Layers** 区域进行关联。

操作步骤

步骤一：创建 Lambda Layer

1. 进入 AWS Lambda 控制台，点击左侧菜单 **Layers**
2. 点击右上角 **Create layer**
3. 填写配置：
 - **Name:** tattooine-common
 - **Upload method:** 选择 *Upload a file from Amazon S3*
 - **S3 link URL:** 填入上方 S3 路径（替换 {REGION}）
 - **Compatible architectures:** 勾选 x86_64
 - **Compatible runtimes:** 选择 Node.js 22.x
4. 点击 **Create**，完成 Layer **Version 1** 的创建

步骤二：将 Layer 关联到两个函数（分别对 FunctionOne 和 FunctionTwo 执行）

1. 进入 **Lambda** → **Functions**，打开目标函数
2. 向下滚动至 **Layers** 区域，点击 **Add a layer**
3. 选择 **Custom layers** → 选择 tattooine-common → 选择 **Version 1**
4. 点击 **Add** 保存

原理说明

Layer 被关联后，Lambda 执行环境会自动将其内容解压至 `/opt/` 目录：

```
/opt/  
  nodejs/  
    node_modules/  
      tattooine-common/ ← 函数可直接 import
```

函数代码无需修改，`require('tattooine-common')` 即可正常导入。

Task 2: 发布 Layer 新版本并灰度升级

背景与问题

tattooine-common 发布了新版本，新库已上传至 S3:

```
s3://aws-jam-challenge-resources-{REGION}/lambda-layers-shared-code/task-two/layer-task-two.zip
```

需要先在 一个函数 上验证新版本，另一个函数暂不升级。

核心概念: Layer 版本不可变性

关键知识点: Lambda Layer 的每个版本是不可变的 (Immutable)。一旦创建，无法修改其内容。要更新 Layer，必须发布一个新版本，每次发布版本号自动递增。

操作步骤

步骤一: 在已有 Layer 上发布新版本

1. 进入 Lambda 控制台 → **Layers**，找到 tattooine-common
2. 点击进入该 Layer 详情页
3. 点击右上角 **Create version**
4. 填写配置:
 - **Description:** v2 - new release (可选)
 - **S3 link URL:** 填入 task-two 的 S3 路径
 - **Compatible architectures:** x86_64
 - **Compatible runtimes:** Node.js 22.x 和 Node.js 18.x

5. 点击 **Create**，生成 **Version 2**

步骤二: 更新 FunctionOne 到 Version 2

1. 打开 ChallengeFunctionOne，进入 **Layers** 区域
2. 点击 **Edit**，先删除已关联的 Version 1 (点击旁边的删除按钮)
3. 再点击 **Add a layer** → Custom layers → tattooine-common → **Version 2**
4. 点击 **Save**

注意：同一个 Layer 的不同版本**不能同时**关联到同一个函数。若不先移除旧版本，直接添加新版本会报错：

Two different versions of the same layer are not allowed to be referenced in the same function.

正确做法是：**先删除旧版本，再添加新版本。**

FunctionTwo 保持关联 Version 1，实现灰度验证。

核心知识点汇总

什么是 Lambda Layers？

Lambda Layers 是一种将**依赖库、自定义运行时、配置或数据文件**与函数代码分离的机制：

- 避免每个函数重复打包相同依赖，减小部署包体积
- 多个函数可共享同一个 Layer，统一管理依赖版本
- 最多可为一个函数关联 **5 个** Layer
- 所有 Layer 内容合并后解压到执行环境的 /opt/ 路径下
- Layer 仅适用于 **.zip 包部署** 的 Lambda 函数；容器镜像函数不使用 Layer 机制

Layer 版本管理

特性	说明
版本不可变	已发布的版本内容无法修改，只能新建版本
版本号递增	每次 Create version，版本号自动 +1（如 v1 → v2 → v3）
ARN 唯一标识	每个版本有独立的 ARN，格式为 arn:aws:lambda:{region}:{account}:layer:{name}:{version}
函数固定版本	函数不会自动跟随 Layer 更新，需手动修改关联的版本号
旧版本不删除	发布新版本后旧版本继续存在，不影响关联了旧版本的函数

Layer 目录结构 (Node.js)

```
layer.zip
  nodejs/
    node_modules/
      your-library/
        index.js
        package.json
```

Lambda 会将此结构解压到 `/opt/`。对于 Node.js，常见可用路径包括：

- `/opt/nodejs/node_modules`
- `/opt/nodejs/node18/node_modules`
- `/opt/nodejs/node20/node_modules`
- `/opt/nodejs/node22/node_modules`

因此，通用的 `nodejs/node_modules` 结构通常最方便；若需要按运行时版本细分，也可使用对应的 `nodeXX` 子目录。

灰度发布策略

通过 Layer 版本可实现分批升级：

1. 发布新 Layer 版本
2. 仅将测试函数切换到新版本
3. 验证无误后，再逐步更新其他函数

这与软件发布中的**金丝雀发布 (Canary Release)** 思路一致。

错误排查速查

错误信息	原因与解决方法
<code>Runtime.ImportModuleError</code>	函数缺少依赖库。创建 Lambda Layer 并关联到函数。
<code>Two different versions of the same layer are not allowed...</code>	同一 Layer 的两个版本同时关联到同一函数。先删除旧版本，再添加新版本。
<code>Layer content is invalid</code>	Layer zip 包目录结构不符合规范。检查 <code>nodejs/node_modules/</code> 路径是否正确。

本文档整理自 AWS Jam 实战挑战 **Lambda Layers Shared Code** 系列题目 | 编写
于 2026 年 3 月 9 日

B.6 CodePipeline Not Working

对应手册

CloudFormation 模板路径故障

适合回看

需要回看 Artifact 路径、ChangeSet Action 和 Jam 保存约束时

PDF 文件

CodePipeline Not Working.pdf

AWS Jam 挑战赛

题目总结与知识点解析

题目名称

CodePipeline 部署故障排查

使用服务：CodeCommit • CodePipeline • CloudFormation

难度 DevOps 中级

涉及服务 AWS CodePipeline, CodeCommit, CloudFormation

核心问题 CloudFormation 模板路径配置错误

解决方式 修正 Deploy 阶段的 Template 文件路径

目录

1 题目背景

贵公司中有人使用 CodePipeline 和 CloudFormation 建立了一条流水线用于部署 AWS 基础设施。该 CodePipeline 包含两个阶段：

- **Source 阶段：**连接一个 CodeCommit 存储库，仓库中包含一个 CloudFormation 模板文件。
- **Deploy 阶段：**使用 Source 阶段获取的模板，部署 CloudFormation 堆栈。

CloudFormation 的部署并未正常进行，Pipeline 处于**失败（Failed）**状态。你被要求作为 DevOps 顾问来排除并解决该问题。

1.1 AWS 资源清单

资源类型	说明
CodeCommit 仓库	包含 CloudFormation 模板的源代码仓库
S3 存储桶	存储 CodePipeline 运行过程中的 Artifact（工件）
IAM 角色	CodePipeline 执行所需的权限角色
CodePipeline	待修复的 CI/CD 流水线

1.2 验收标准

- CodePipeline 执行成功完成
- 新的 CloudFormation 堆栈成功部署
- CloudFormation 堆栈名称出现在主堆栈的 Outputs 中

2 故障排查过程

2.1 第一步：查看错误信息

进入 AWS 控制台 → CodePipeline → 点击失败的 Pipeline → 找到失败阶段 → 点击 **Details**。

错误信息：

```
File [template.yaml] does not exist in artifact [SourceApp]
```

错误含义解析：

- Deploy 阶段在名为 SourceApp 的 Artifact 中寻找文件 `template.yaml`
- 该文件在 Artifact 中**不存在**，导致部署失败
- 根本原因：Pipeline 配置的模板路径与仓库中实际的文件路径不匹配

2.2 第二步：定位根本原因

配置项	错误配置	正确配置
文件名	<code>template.yaml</code>	<code>working.template.yaml</code>
文件路径	根目录	<code>infrastructure-as-code/</code> 子目录
完整路径	<code>template.yaml</code>	<code>infrastructure-as-code/working.template</code>

仓库实际目录结构如下：

```
/ (仓库根目录)
  infrastructure-as-code/
    working.template.yaml  <- 实际文件位置
```

3 解决操作步骤

重要提示：在本 Jam 环境中，保存 Pipeline 修改时需要勾选“No resource updates needed for this source action change”，否则控制台可能因尝试更新变更检测相关资源而报权限错误。

这不是所有生产环境都必然需要的步骤，而是当前实验权限模型下的特殊约束。

Step 1. 进入 AWS 控制台 → CodePipeline，选择失败的 Pipeline。

Step 2. 点击右上角 **Edit**（编辑）按钮。

Step 3. 找到 **DeployTestStack** 阶段，点击 **Edit stage**。

Step 4. 点击 **GenerateChangeSet** Action 旁的编辑图标（铅笔图标）。

Step 5. 在 **Template** 部分，将 **File name** 字段修改为：

```
infrastructure-as-code/working.template.yaml
```

Step 6. 点击 **Done**（忽略顶部可能出现的 **ValidationError** 提示）。

Step 7. 点击 **DeployTestStack** 阶段的 **Done**。

Step 8. 点击顶部 **Save** 按钮。

Step 9. 弹出提示框后：

- 勾选 “No resource updates needed for this source action change”
- 点击 **Save** 确认保存

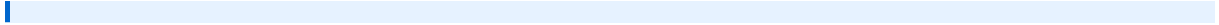
Step 10. 如果 Pipeline 未自动触发，点击 **Release Change** → 弹窗中点击 **Release**。

Artifact 路径格式说明：

在 CodePipeline Deploy 阶段配置 CloudFormation 模板路径时，格式为：

```
ArtifactName:: 相对路径/文件名.yaml
# 例如：
SourceApp::infrastructure-as-code/working.template.yaml
```

路径必须与 CodeCommit 仓库中的实际文件路径完全一致。



4 知识点解析

4.1 AWS CodePipeline 概述

4.1.1 定义

AWS CodePipeline 是一项全托管的 CI/CD（持续集成/持续交付）服务，用于自动化代码从提交到部署的完整流程。

4.1.2 核心概念

概念	说明
Pipeline（流水线）	由多个阶段组成的自动化 workflow
Stage（阶段）	流水线中的一个逻辑步骤，如 Source、Build、Deploy
Action（动作）	每个阶段中执行的具体任务
Artifact（工件）	各阶段之间传递数据的“包裹”，存储于 S3

4.1.3 典型流水线结构

```
Source --> Build --> Test --> Deploy
源码      构建      测试      部署
(CodeCommit) (CodeBuild) (测试工具) (CloudFormation/ECS)
```

本题的流水线结构（简化版）：

```
Source (CodeCommit) --> Deploy (CloudFormation)
```

4.2 CodePipeline 与相关服务的关系

服务	职责
CodeCommit	代码仓库，类似 GitHub/GitLab
CodeBuild	构建和测试代码，类似 Jenkins
CodeDeploy	将应用部署到 EC2、ECS 等计算资源
CloudFormation	基础设施即代码，自动化创建 AWS 资源
CodePipeline	编排器：串联上述服务，按顺序自动触发

关键理解： CodePipeline 本身不执行具体任务，它是一个调度器和编排器，负责按顺序触发各阶段的工具并在它们之间传递 Artifact。

4.3 CloudFormation 在 CodePipeline 中的使用

当 Deploy 阶段使用 CloudFormation 时，常见的 Action 类型有：

Action 类型	说明
Create or Update Stack	直接创建或更新 CloudFormation 堆栈
Create Change Set	生成变更集（本题使用的 GenerateChangeSet）
Execute Change Set	执行已生成的变更集
Delete Stack	删除 CloudFormation 堆栈

4.4 常见 CodePipeline 失败原因

失败阶段	常见原因	解决方案
Source	分支名称错误	核对 CodeCommit 实际分支名
Source	仓库访问权限不足	检查 IAM 角色权限
Deploy	模板路径错误	核对仓库文件实际路径（本题）
Deploy	IAM 权限不足	为 CodePipeline 角色添加 CF 权限
Deploy	S3 Artifact 桶跨区域	确保桶与 Pipeline 在同一区域

5 经验总结

5.1 排查流程

```
Pipeline 报错
|
v
查看失败阶段的 Details 错误信息
|
v
"File does not exist in artifact"?
|
v
去 CodeCommit 查看仓库实际文件结构
(或通过 S3 Artifact 桶下载 zip 查看)
|
v
确认完整相对路径 + 文件名
|
v
修改 Deploy 阶段 Template 路径
|
v
保存 (勾选无需资源更新) -> Release Change
```

5.2 最佳实践

- 文件命名规范: CloudFormation 模板使用清晰的命名, 避免与默认名称混淆
- 路径验证: 配置 Pipeline 后, 先确认 Source 阶段能成功拉取文件, 再调试 Deploy 阶段
- 权限最小化: CodePipeline IAM 角色只授予必要权限, 遵循最小权限原则
- 多环境隔离: 为 Dev/Staging/Prod 分别创建独立的 Pipeline, 避免相互干扰
- 变更集审查: 生产环境部署前使用 Change Set 预览变更内容, 再执行

5.3 本题的局限性说明

实验环境限制：本 Jam 题目限制了 `codecommit:ListRepositories` 等权限，导致正常的排查路径（浏览仓库文件结构）无法执行。这是实验环境的人为约束；在真实企业环境中，DevOps 工程师通常至少会具备以下之一：

- CodeCommit 仓库读取权限
- S3 Artifact 桶完整访问权限
- 可向开发团队询问文件结构

本题的核心考察点是：**知道在 Pipeline 的哪个位置修改模板路径**，而非独立推导出完整路径。

B.7 NEED FOR SPEED!!!

对应手册

CodePipeline 并行加入 Lint

适合回看

需要回看 Analysis Stage、runOrder 和预建资源时

PDF 文件

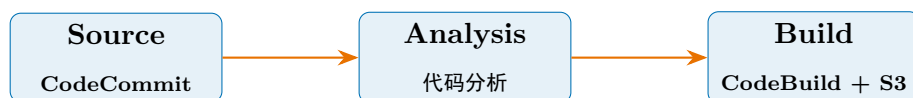
NEED FOR SPEED!!!.pdf

AWS Jam 挑战题总结

CodePipeline 集成代码检查（Lint）

题目背景

某公司已构建了一个用于生成应用程序代码的 AWS CodePipeline，流水线分为三个顺序阶段：



- **Source（来源）**：CodeCommit 仓库，存储源代码
- **Analysis（分析）**：CodeBuild 执行代码分析
- **Build（构建）**：CodeBuild 构建，产物存储至 S3

挑战任务

注意

在不增加构建时间的前提下，向现有 CodePipeline 中添加代码检查（Lint）规范强制执行机制。

挑战环境已预先创建的资源：

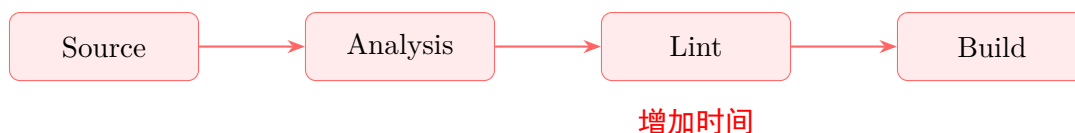
- CodeCommit 仓库（含源码和 buildspec 文件）
- S3 存储桶（存储 Pipeline 工件）
- CodePipeline 的 IAM 角色及内联策略

- CodePipeline 实例
- CodeBuild 项目（含预建的 LintProject）

核心思路

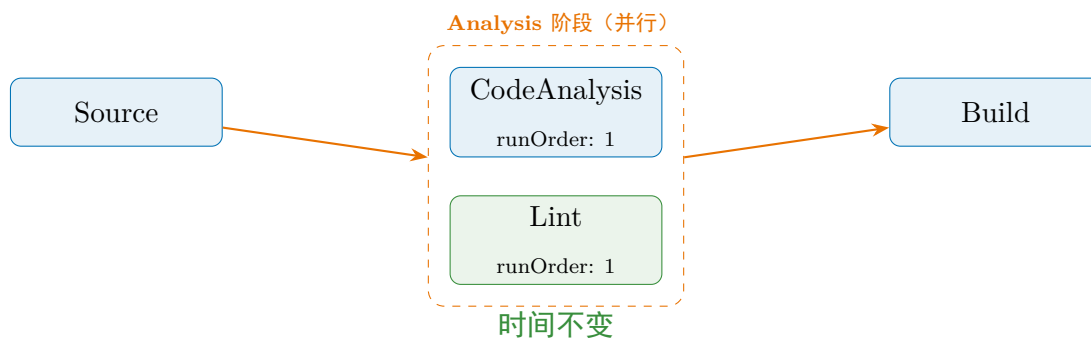
问题分析

若将 Lint 作为独立的新阶段串行添加，必然增加整体构建时间：



解决方案：同阶段并行执行

将 Lint 动作加入 **Analysis** 阶段，与现有分析动作并行运行（runOrder 设为相同值）：



关键结论

CodePipeline 同一 Stage 内，runOrder 相同的多个 Action 会并行执行。这意味着耗时不会像串行阶段那样直接相加，但整个 Stage 的完成时间仍取决于最慢的那个 Action。

操作步骤

第一步：进入 CodePipeline 控制台，编辑流水线

1. 打开 AWS Console → CodePipeline → 选择目标 Pipeline
2. 点击右上角 编辑
3. 找到 **Analysis** 阶段，点击 添加操作

第二步：配置 Lint 操作（填写编辑操作表单）

字段	填写值	说明
操作名称	Lint	自定义名称
操作提供程序	AWS CodeBuild	由 CodeBuild 执行
区域	美国（弗吉尼亚北部）	与 Pipeline 同区域
输入构件	SourceArtifact	使用源码作为输入
项目名称	LintProject-xxx	选择预建的 Lint 项目
构建类型	单次构建	触发一次构建实例

关键配置

确保 Lint Action 与 CodeAnalysis Action 的 **运行顺序（Run Order）** 均为 1，这是实现并行执行的核心。

第三步：保存并发布更改

1. 点击 **完成** 保存操作配置
2. 点击 **保存** 保存 Pipeline
3. 在“发布更改”对话框中，**勾选**“此更新无需资源更新”
4. 点击确认，Pipeline 自动触发执行

知识点详解

CodePipeline 核心概念

操作提供程序（Action Provider）

知识点

CodePipeline 本身只是**调度器**，不执行具体任务，需要指定提供程序来真正干活。

提供程序	用途
AWS CodeBuild	运行构建、测试、Lint 等脚本
AWS CodeDeploy	部署应用到服务器
AWS Lambda	执行自定义无服务器函数
Amazon S3	上传/下载工件文件

输入构件（Input Artifact）

每个 Stage 会产出一个”工件包裹”，下游 Action 通过输入构件声明使用哪个包裹作为原料：

```

1 Source 阶段 --产出--> SourceArtifact（源代码压缩包）
2                               |
3                               v
4 Analysis/Lint 阶段 <--输入-- SourceArtifact（直接分析源码）
5                               |
6                               v
7 Build 阶段 <--输入-- AnalysisOutput（上阶段产出）
  
```

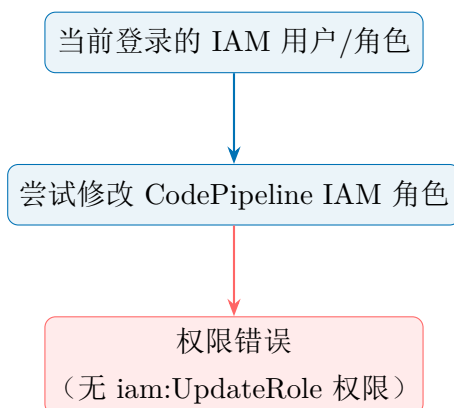
Listing 1: 构件流转示意

构建类型（Build Type）

类型	说明	适用场景
单次构建	启动 1 个 CodeBuild 实例	普通构建/Lint
批处理构建	同时启动多个实例	多平台/多版本并行测试

”此更新无需资源更新”选项

为什么不勾选会报权限错误？



为什么勾选后正常工作？

知识点

挑战环境已预先配置好所有 IAM 权限，无需 AWS 自动更新角色。勾选该选项等于告诉 AWS：“IAM 已经 OK，跳过角色更新步骤。”

```
1 # 不勾选（报错）
2 Pipeline 保存 -> AWS 尝试更新 IAM 角色 -> 无权限 -> 失败
3
4 # 勾选（成功）
5 Pipeline 保存 -> 跳过 IAM 更新 -> 直接保存 -> 成功
```

Listing 2: 两种情况对比

CodeBuild 与 Lint 的关系

知识点

AWS CodeBuild 没有内置 Lint 工具，但提供了标准化的运行环境。通过在 `buildspec.yml` 中安装和调用 Lint 工具来实现代码检查。

```
1 version: 0.2
2
3 phases:
4   install:
5     runtime-versions:
6       python: 3.11
7     commands:
8       - pip install pylint --quiet # 安装 Lint 工具
9
10  pre_build:
11    commands:
12      - echo "Running lint check..."
13      - pylint src/**/*.py # 执行 Lint
14
15  build:
16    commands:
17      - echo "Building application..."
18
19  artifacts:
20    files:
21      - '**/*'
```

Listing 3: buildspec.yml 中集成 Lint 示例

常用 Lint 工具对照：

语言	Lint 工具	安装命令
Python	pylint / flake8	<code>pip install pylint</code>
JavaScript	ESLint	<code>npm install -g eslint</code>
Ruby	RuboCop	<code>gem install rubocop</code>
Go	golint	<code>go install</code> <code>golang.org/x/lint</code>

成功验证标准

检查项	预期结果
Pipeline 结构	Analysis 阶段显示两个并行 Action
构建时间	与原来相比无明显增加
Lint 失败时	Pipeline 在 Analysis 阶段停止, 不进入 Build
挑战标记	CodePipeline 成功执行后自动标记挑战完成

总结

关键结论

核心知识点一句话概括:

利用 CodePipeline 同一 Stage 内 `runOrder` 相同的 Action 并行执行的特性, 将 Lint 与现有 Analysis 同步运行, 避免串行增加时间; 若 Lint 不比现有分析更慢, 通常不会明显拉长 Stage。

IAM 权限一句话概括:

挑战环境预配置了所有权限, 勾选“此更新无需资源更新”跳过不必要的 IAM 角色更新, 避免因缺少 `iam:UpdateRole` 权限而报错。

B.8 Protect Your ECR Image

对应手册

ECR 资源策略修复

适合回看

需要回看 Deny 条件逻辑、CodeBuild 角色 ARN 和 ECR 推送权限时

PDF 文件

Protect Your ECR Image.pdf

AWS Jam Challenge

Amazon ECR 资源策略修复

知识点总结 & 操作指南

涉及服务： Amazon ECR | AWS IAM | AWS CodeBuild

文档生成日期：2026 年 3 月 9 日

目录

1 题目背景

1.1 场景描述

在一个典型的 CI/CD 流水线中，**AWS CodeBuild** 负责构建 Docker 镜像并将其推送至 **Amazon ECR**（弹性容器注册表）。然而，由于 ECR 仓库配置了**资源策略（Resource-based Policy）**，其中存在一条 **Deny** 规则，导致 CodeBuild 的服务角色也被拦截，无法执行推送操作。

1.2 问题仓库

ECR 仓库名称: pyei-challenge-app

1.3 原始策略（有问题的配置）

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "DenyPutImage",
6       "Effect": "Deny",
7       "Principal": "*",
8       "Action": "ecr:PutImage"
9     }
10  ]
11 }
```

问题所在：该策略对 **Principal: "*" 全体主体拒绝 ecr:PutImage 操作**，没有任何例外，导致 CodeBuild 的服务角色（Service Role）也被拒绝推送镜像。

2 任务目标

修改 **DenyPutImage** 语句，使用 **条件键（Condition Key）** **aws:PrincipalArn** 排除 AWS CodeBuild 服务角色，使其不受 Deny 规则影响，从而恢复 CI/CD 流水线的正常推送能力。

验收标准：在 ECR 资源策略的 Deny 语句中，使用 `aws:PrincipalArn` 全局条件键排除 CodeBuild 服务角色 ARN。

3 操作步骤

3.1 第一步：获取 CodeBuild 服务角色 ARN

1. 登录 AWS 管理控制台
2. 导航至 IAM → Roles（角色）
3. 在搜索框输入 `codebuild` 进行筛选
4. 点击对应的 CodeBuild 服务角色
5. 复制其完整 ARN

ARN 格式示例：

```
arn:aws:iam::< 账号 ID>:role/codebuild-< 项目名 >-service-role
```

3.2 第二步：修改 ECR 资源策略

1. 进入 Amazon ECR 控制台
2. 在左侧导航栏选择 **Repositories**（仓库）
3. 点击仓库 `pyei-challenge-app`
4. 选择 **Permissions**（权限）标签页
5. 点击 **Edit policy JSON**
6. 将策略替换为修复后的版本（见第四节）
7. 点击 **Save**（保存）

3.3 第三步：验证修复结果

在 CI/CD 流水线中重新触发构建，或通过以下 AWS CLI 命令测试推送：

```
1 # 登录到 ECR
2 aws ecr get-login-password --region <region> | \
3     docker login --username AWS \
4         --password-stdin <account_id>.dkr.ecr.<region>.amazonaws.com
5
6 # 推送镜像
7 docker push <account_id>.dkr.ecr.<region>.amazonaws.com/pyei-
    challenge-app:latest
```

4 修复后的策略

4.1 完整策略 JSON

```
1 {
2     "Version": "2012-10-17",
3     "Statement": [
4         {
5             "Sid": "DenyPutImage",
6             "Effect": "Deny",
7             "Principal": "*",
8             "Action": "ecr:PutImage",
9             "Condition": {
10                 "ArnNotEquals": {
11                     "aws:PrincipalArn":
12                         "arn:aws:iam::123456789012:role/codebuild-xxx-service-
13                         role"
14                 }
15             }
16         ]
17     }
```

注意：请将 `arn:aws:iam::123456789012:role/codebuild-xxx-service-role` 替换为你在第一步中找到的真实 CodeBuild 服务角色 ARN。

5 核心知识点

5.1 IAM Policy 评估逻辑

AWS IAM 在评估权限时遵循以下优先级规则：

优先级	规则	结果
1	显式 Deny（无条件）	拒绝
2	显式 Deny（条件不满足时跳过）	取决于条件
3	显式 Allow	允许
4	隐式 Deny（无匹配语句）	拒绝

5.2 全局条件键 `aws:PrincipalArn`

`aws:PrincipalArn` 是一个 IAM 全局条件键（Global Condition Key），用于匹配发出请求的 IAM 主体（Principal）的 ARN。

条件运算符	含义	适用场景
<code>ArnEquals</code>	ARN 完全匹配	精确排除某个角色
<code>ArnNotEquals</code>	ARN 不匹配时为真	排除特定角色
<code>ArnLike</code>	ARN 支持通配符匹配	排除一类角色
<code>ArnNotLike</code>	ARN 不匹配通配符时为真	排除一类角色

5.3 Deny + ArnNotEquals 的组合逻辑

这是本题的核心逻辑，可用以下表格理解：

请求者	<code>ArnNotEquals</code> 结果	Condition 满足？	Deny 生效？
CodeBuild 角色	false	否	否（允许）
其他 IAM 主体	true	是	是（拒绝）

理解要点：条件（Condition）是 Deny 语句的触发门槛。只有当条件为 `true` 时，Deny 才生效。使用 `ArnNotEquals` 意味着：“除了指定的 ARN 以外的所有人，才触发拒绝”。

5.4 ECR 资源策略 vs IAM 身份策略

属性	ECR 资源策略	IAM 身份策略
附加对象	ECR 仓库（资源）	IAM 用户/角色/组
控制方向	谁可以访问此资源	此主体可访问哪些资源
跨账号访问	支持	需配合资源策略
Principal 字段	必须指定	不需要（隐式为附加主体）

5.5 Amazon ECR 常用操作权限

权限	用途
ecr:PutImage	创建或更新镜像 manifest 与 tag
ecr:GetDownloadUrlForLayer	拉取镜像层
ecr:BatchGetImage	批量获取镜像
ecr:GetAuthorizationToken	获取认证 Token（需 IAM 策略）
ecr:InitiateLayerUpload	初始化镜像层上传
ecr:UploadLayerPart	上传镜像层分片
ecr:CompleteLayerUpload	完成镜像层上传

6 架构图解

6.1 CI/CD 流水线与 ECR 交互流程

```

开发者提交代码
|
v
[AWS CodePipeline]
|
v
[AWS CodeBuild] <-- 使用 Service Role ARN
|
| docker push
v
[Amazon ECR]
pyei-challenge-app
|

```

| ECR 资源策略检查

v

DenyPutImage 语句

Condition: ArnNotEquals

CodeBuild Role ARN → 条件为false

→ Deny 不生效 → 允许推送

7 常见错误与排查

Q1. 使用了错误的条件键

错误：使用 `aws:PrincipalType` 或 `aws:userid`

正确：使用 `aws:PrincipalArn`，这是按角色 ARN 做精确匹配时最直接、最清晰的全局条件键。

Q2. ARN 填写不完整

错误：只填写角色名 `codebuild-role`

正确：必须填写完整 ARN，如 `arn:aws:iam::123456789:role/codebuild-role`

Q3. 条件运算符选择错误

错误：使用 `ArnEquals`（这会导致只有 CodeBuild 被拒绝）

正确：使用 `ArnNotEquals`（排除 CodeBuild，其余全部拒绝）

Q4. 忘记保存策略

修改完 JSON 后，必须点击 **Save** 按钮，否则修改不会生效。

8 总结

核心要点回顾：

1. ECR 资源策略的 Deny 语句可以通过 **Condition** 添加例外
2. 使用全局条件键 `aws:PrincipalArn` 匹配请求者的 ARN
3. `ArnNotEquals` + Deny 组合 = “除了指定 ARN 以外，全部拒绝”
4. 修复 CI/CD 流水线权限问题时，**最小权限原则**是首选方案
5. 排查 ECR 推送失败时，需同时检查 **ECR 资源策略**和 **IAM 身份策略**

AWS Jam Challenge 知识点总结文档
基于 Amazon ECR 资源策略排错场景

本手册基于当前目录下 8 份 AWS Jam 文档重组而成。